

NANYANG TECHNOLOGICAL UNIVERSITY

Data Structures & Algorithms in Python

Searching – Lab Answers

College of Computing and Data Science

Recap

- Complexity
- O()?
- Ω()?
- Θ()?

Content Copyright Nanyang Technological University

Searching

Sequential search

- O(n)

55	8	13	1	89	34	2	21	55	3

1	2	3	5	8	13	21	34	55	89

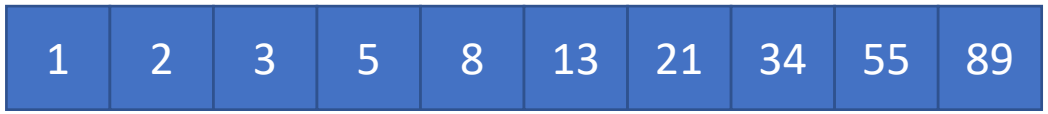


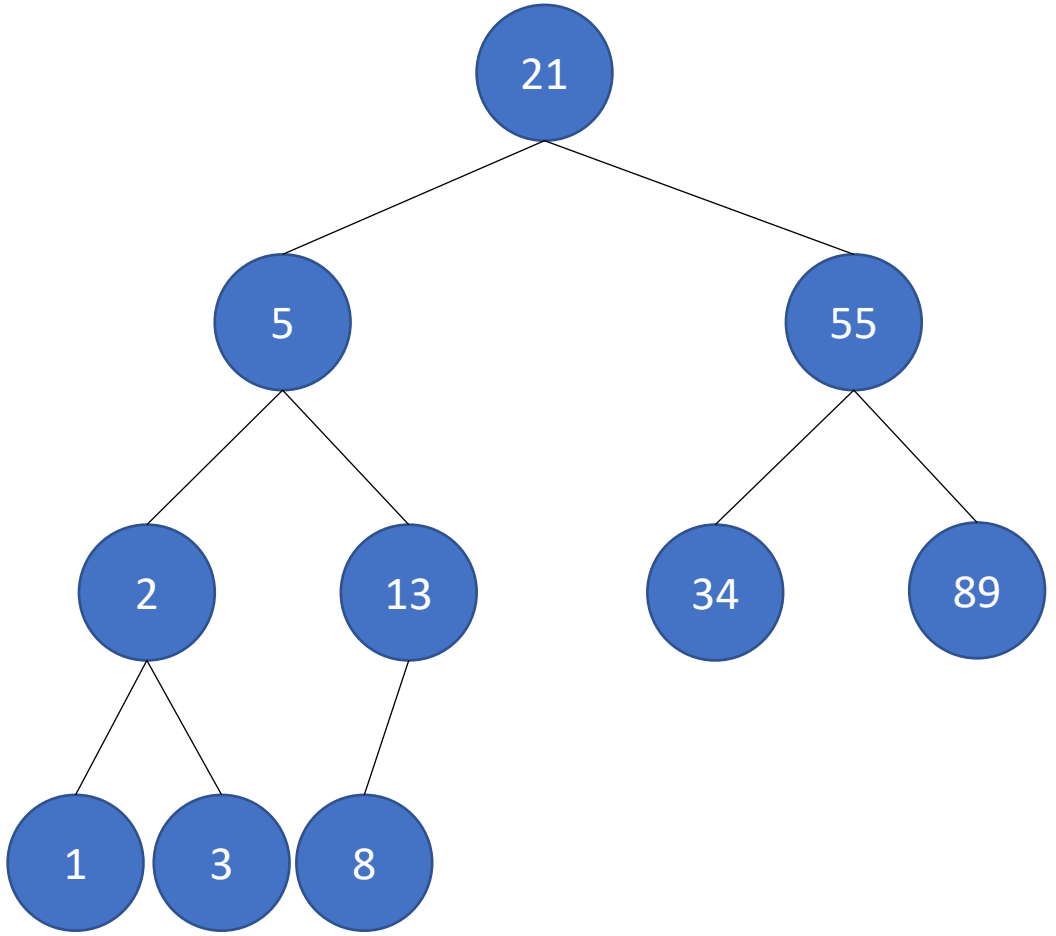
Content Copyright Nanyang Technological University

Searching

Binary search

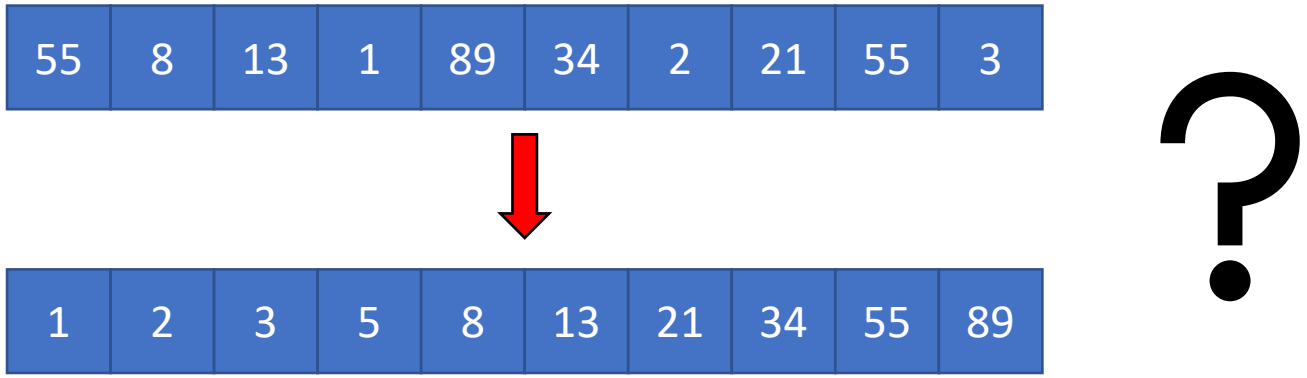
- O(log n)





Content Copyright Nanyang Technological University

Searching



Merge sort + binary search
O(n log n) + O(log n)
vs
Sequential search
O(n)

Content Copyright Nanyang Technological University

Searching

Search k times

- Sequential search: O(k*n) → $O(n^2)$
- Merge sort + Binary search: O(n log n) + O (k log n) → O(n log n)

Content Copyright Nanyang Technological University

Searching

Jump search

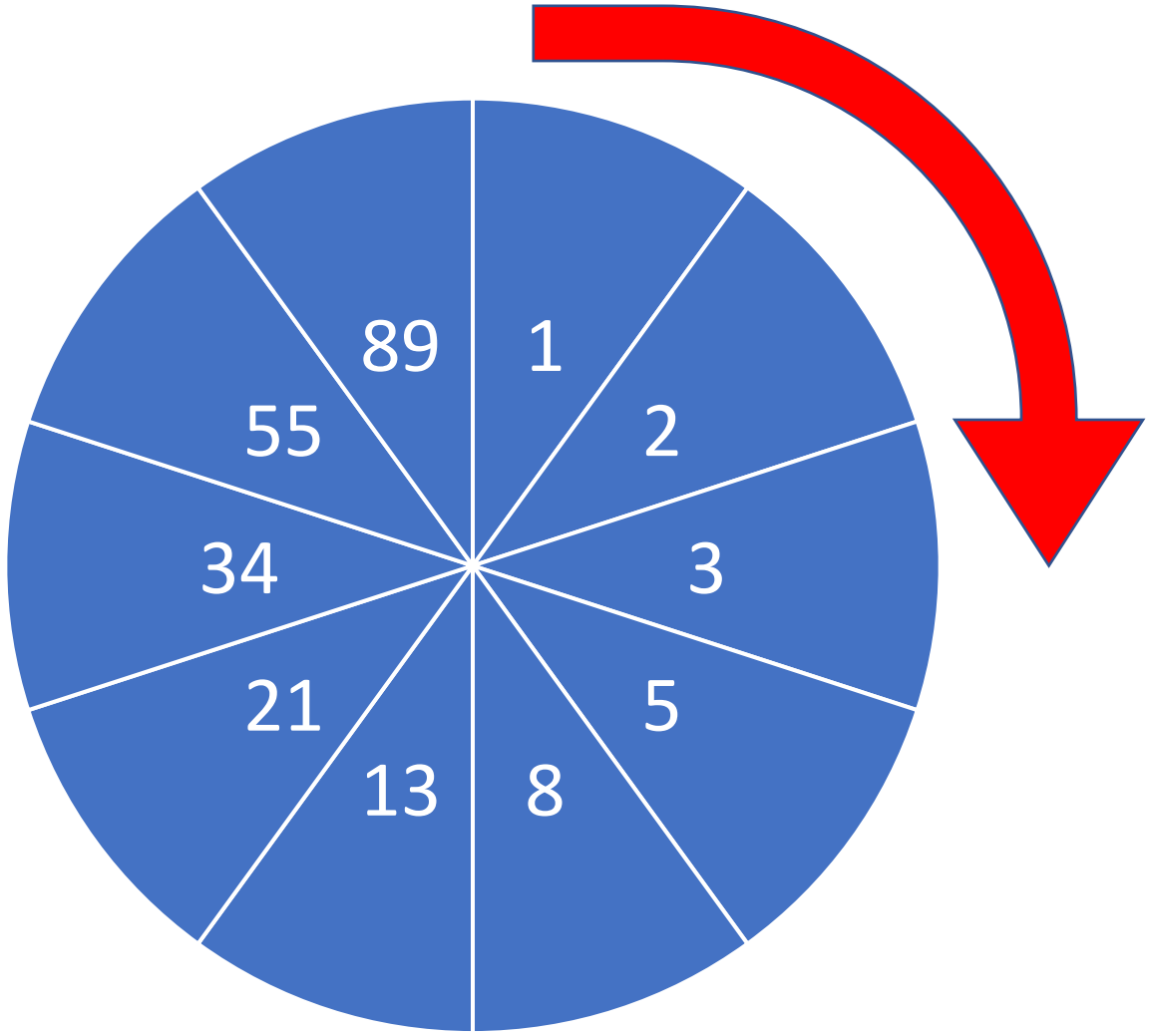
- O(√n)

Why jump over binary?

1	2	3	5	8	13	21	34	55	89

target = 2

Binary search → 3 rotations



Content Copyright Nanyang Technological University

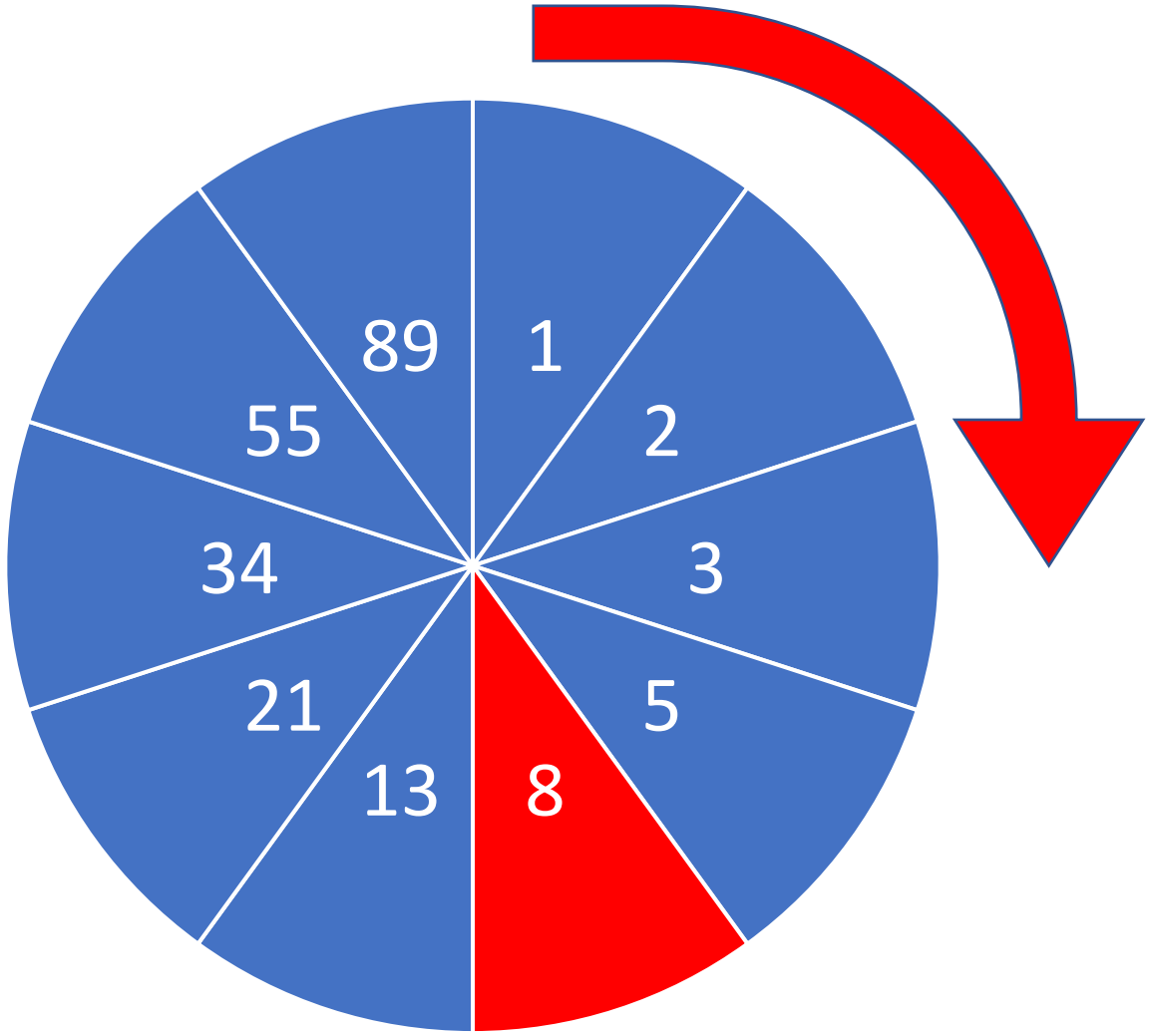
Searching

Why jump over binary?

1	2	3	5	8	13	21	34	55	89

target = 2

Binary search → 3 rotations



Content Copyright Nanyang Technological University

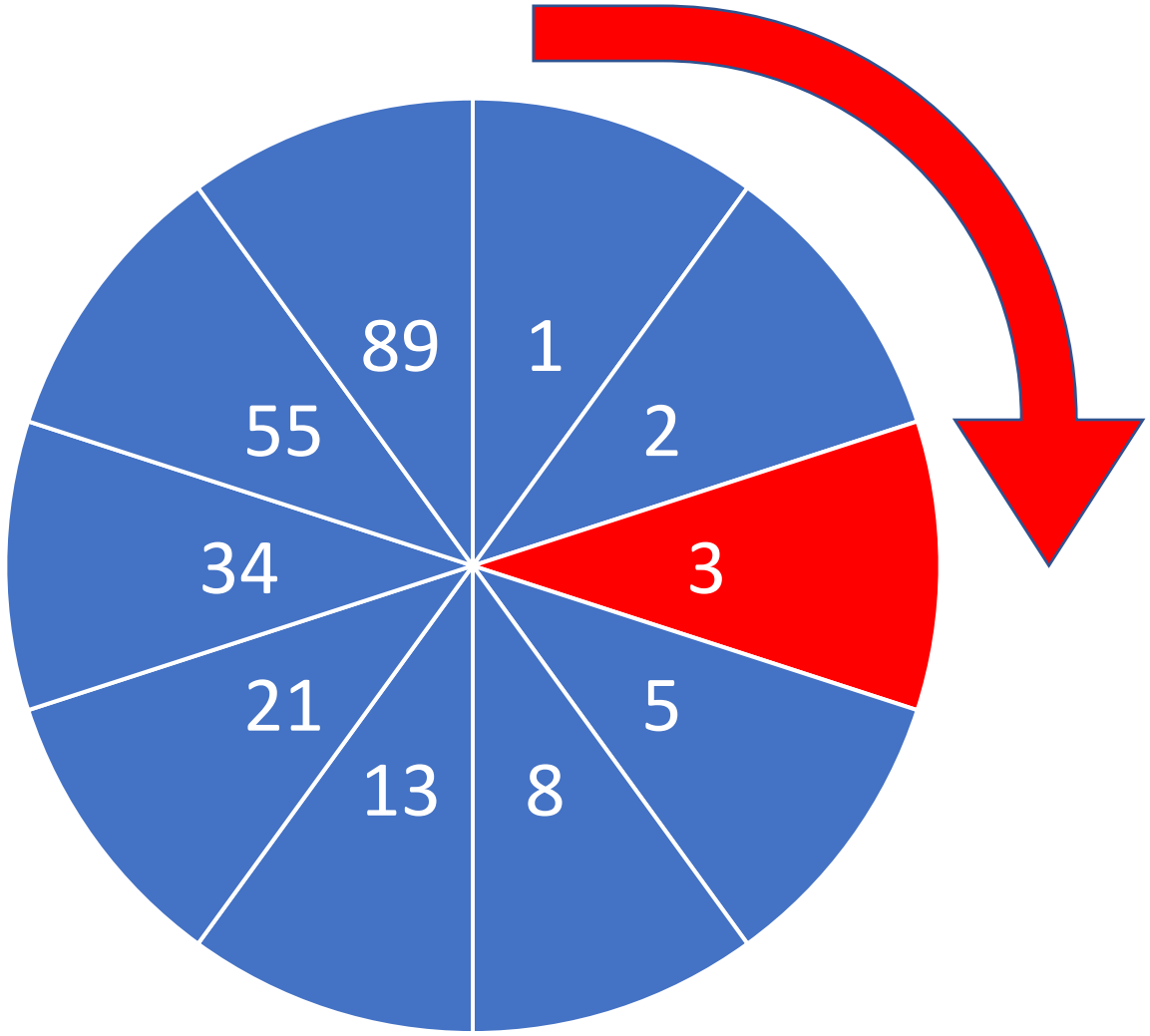
Searching

Why jump over binary?

1	2	3	5	8	13	21	34	55	89

target = 2

Binary search → 3 rotations



Content Copyright Nanyang Technological University

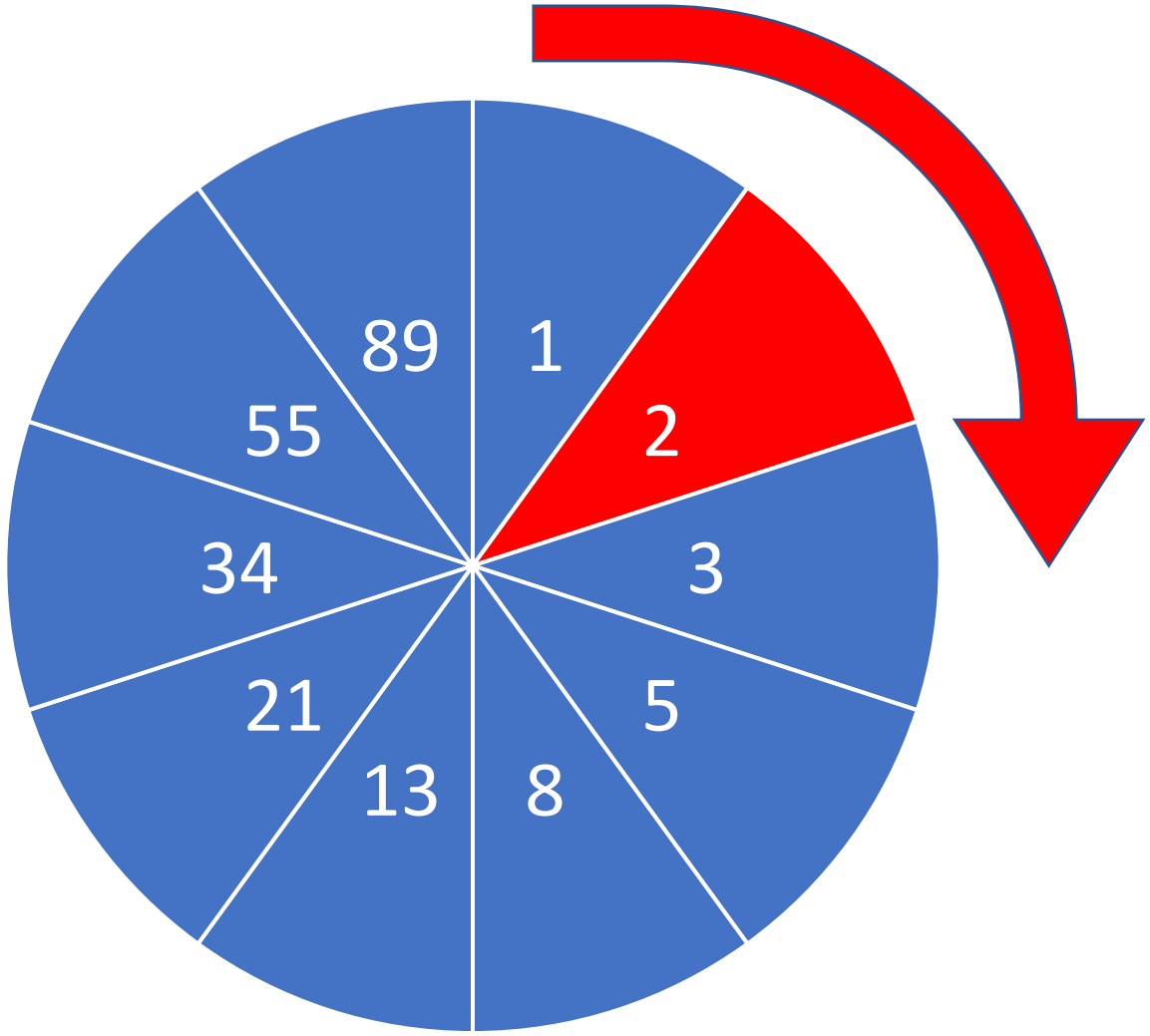
Searching

Why jump over binary?

1	2	3	5	8	13	21	34	55	89

target = 2

Binary search → 3 rotations



Content Copyright Nanyang Technological University

Searching

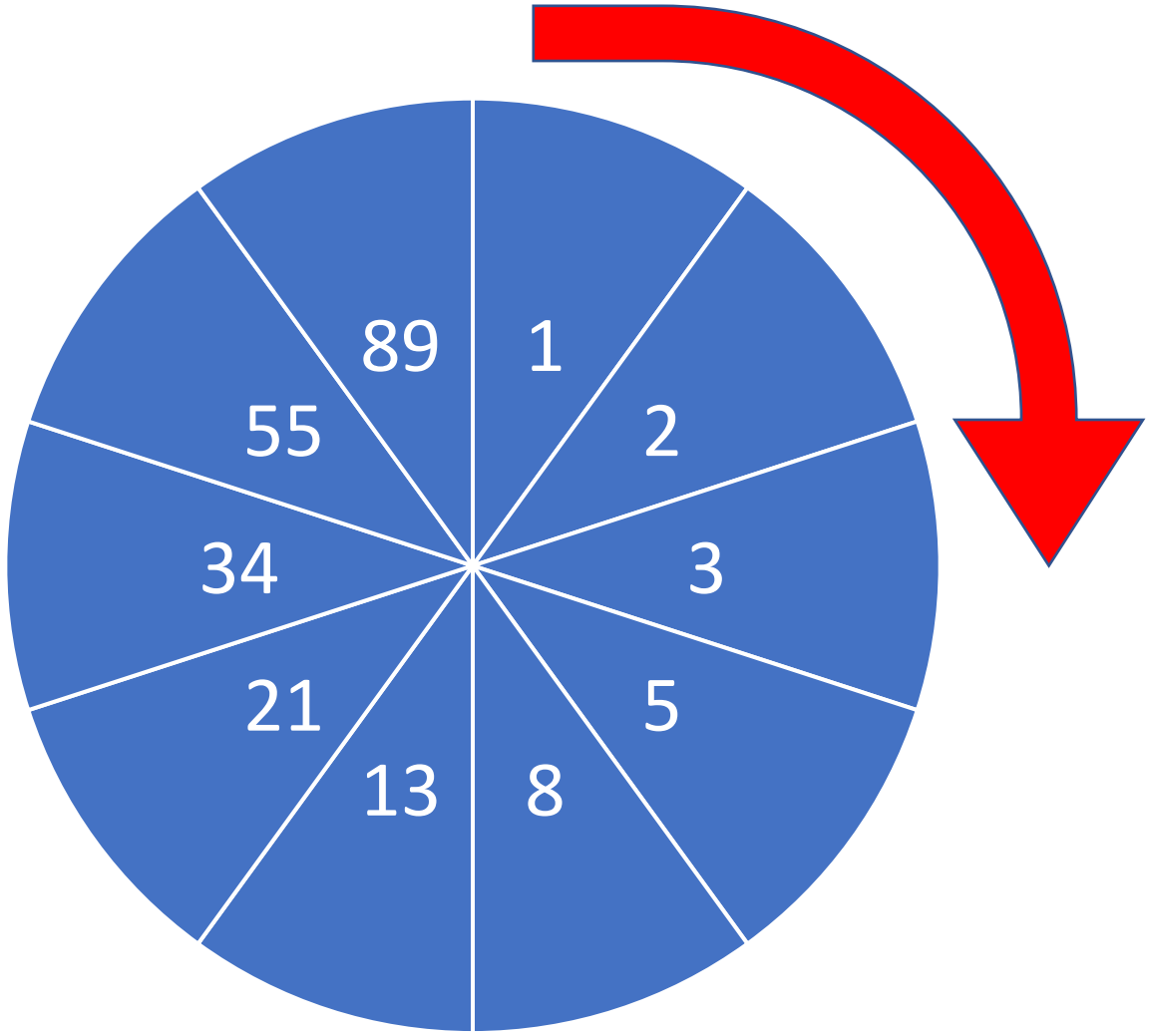
Why jump over binary?

1	2	3	5	8	13	21	34	55	89

target = 2

Binary search → 3 rotations

Jump search → 2 rotations



Content Copyright Nanyang Technological University

Searching

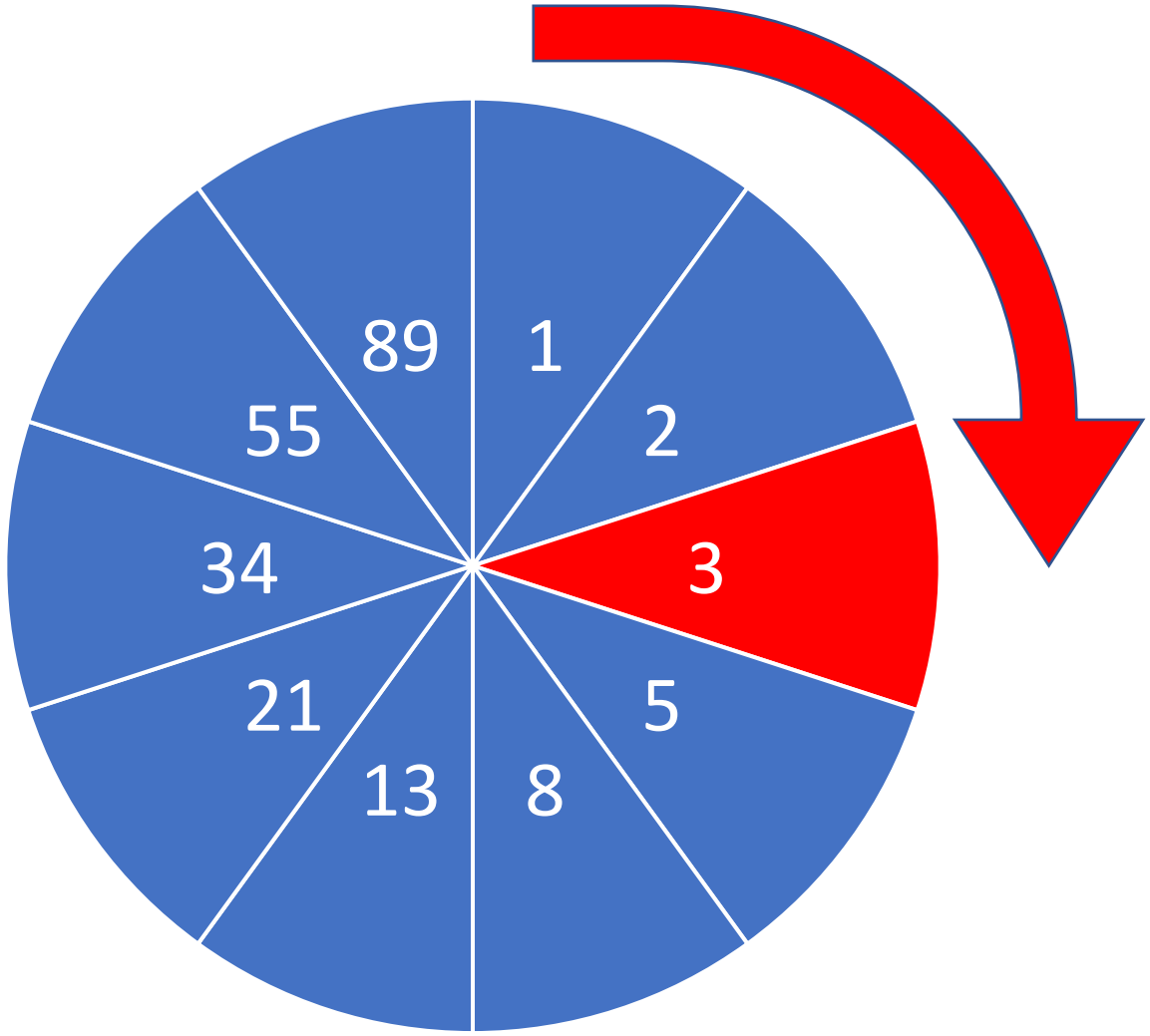
Why jump over binary?

1	2	3	5	8	13	21	34	55	89

target = 2

Binary search → 3 rotations

Jump search → 2 rotations



Content Copyright Nanyang Technological University

Searching

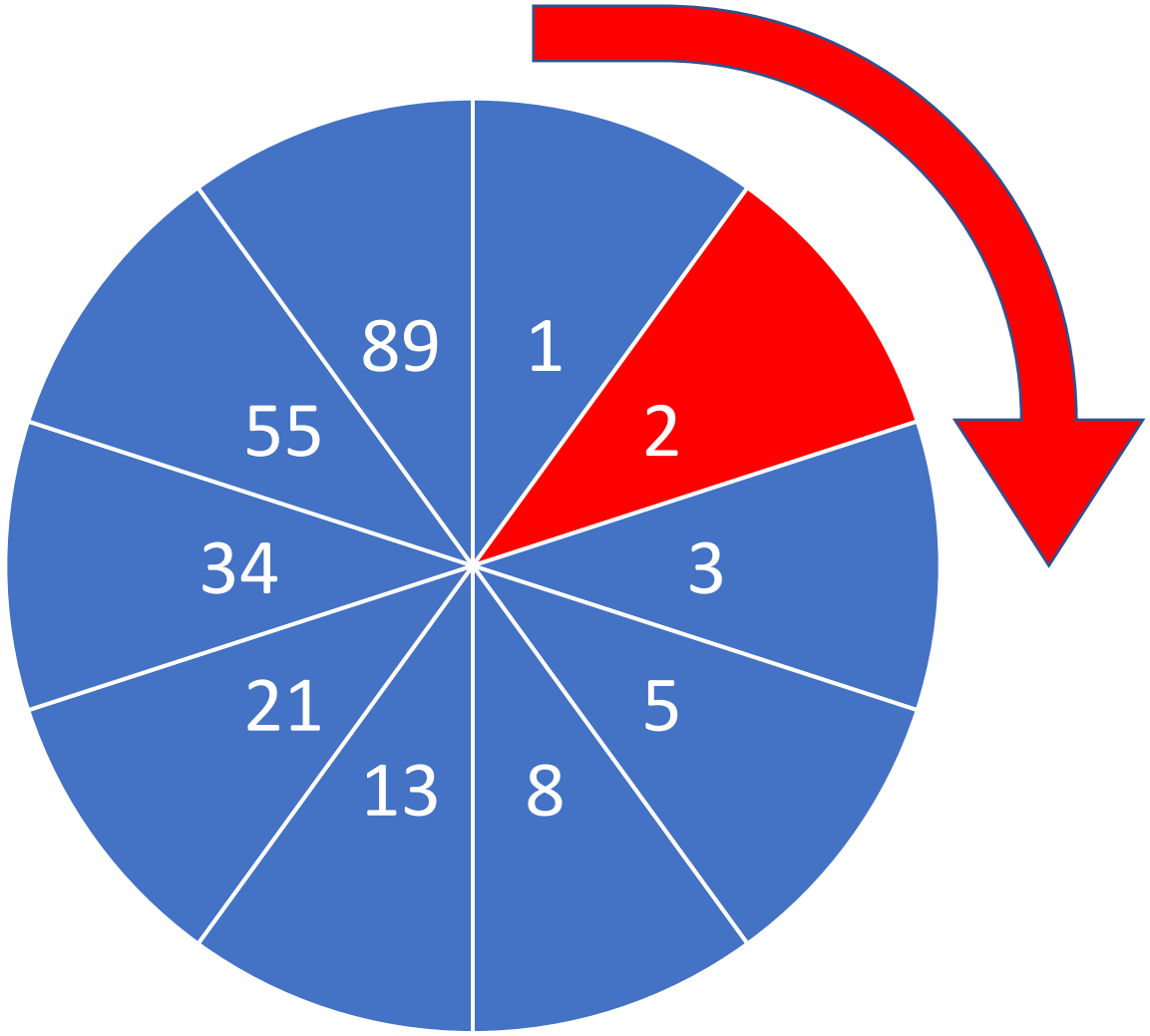
Why jump over binary?

1	2	3	5	8	13	21	34	55	89

target = 2

Binary search → 3 rotations

Jump search → 2 rotations



Content Copyright Nanyang Technological University

Q1 (No AI)

Given an array A of n integers and a target value K, find two elements of A whose sum is equal to K. The two elements can be the same element, so a pair may use the same index twice (A[i] + A[i]). Once a pair is found, the search can stop.

To make the answer unique, follow these rules:

- Return the pair as a list [i, j] with i <= j.
- If more than one pair adds up to K, return the one with the smallest i. If several pairs share that i, return the one with the smallest j.
- If no pair adds up to K (including when A is empty), return [-1, -1].

Input

A (list of int) — the input array of integers, not necessarily sorted

K (int) — the target sum

For example, given A = [3, 1, 7, 4, 5, 9], K = 8:

Three pairs add up to 8: A[0] + A[4] = 3 + 5, A[1] + A[2] = 1 + 7 and A[3] + A[3] = 4 + 4

The pair starting at index 0 has the smallest first index

Output

A list of two indices [i, j] such that A[i] + A[j] == K, or [-1, -1] if there is no such pair.

For the example above:

[0, 4]

3 + 5 = 8, and no pair has a first index smaller than 0.

Content Copyright Nanyang Technological University

Q2 (Normal)

Given an array A of n integers sorted in ascending order, find two elements of A whose sum is equal to K. The two elements can be the same element, so a pair may use the same index twice (A[i] + A[i]).

Because A is sorted, you do not need to check every pair: your algorithm must run in O(n) time.

To make the answer unique, follow these rules:

- All values in A are different.
- Return the pair as a list [i, j] with i <= j.
- If more than one pair adds up to K, return the one with the smallest i.
- If no pair adds up to K (including when A is empty), return [-1, -1].

Input

A (list of int) — a sorted array of distinct integers

K (int) — the target sum

For example, given A = [1, 3, 4, 5, 7, 9], K = 8:

Three pairs add up to 8: A[0] + A[4] = 1 + 7, A[1] + A[3] = 3 + 5 and A[2] + A[2] = 4 + 4

The pair starting at index 0 has the smallest first index

Output

A list of two indices [i, j] such that A[i] + A[j] == K, or [-1, -1] if there is no such pair.

For the example above:

[0, 4]

1 + 7 = 8, and no pair has a first index smaller than 0.

Content Copyright Nanyang Technological University

Q3 (Driver)

Given two arrays num1 and num2, each sorted in ascending order, with lengths m and n, find the median of all m + n numbers combined. Your solution must run in O(log(min(m, n))) time, so merging the two arrays is not enough.

The median is the middle value of the combined sorted numbers:

- If m + n is odd, it is the middle element.
- If m + n is even, it is the average of the two middle elements (for example 2.5).

Input

num1 (list of int) — the first sorted array, possibly empty

num2 (list of int) — the second sorted array, possibly empty

At least one of the two arrays is non-empty.

For example, given num1 = [1, 3, 8], num2 = [7, 9, 10, 11]:

Combined and sorted, the numbers are [1, 3, 7, 8, 9, 10, 11]

There are 7 numbers (odd), so the median is the 4th one

Output

A single number: the median of the combined numbers.

For the example above:

8

The 4th of the 7 combined numbers is 8.

Content Copyright Nanyang Technological University

Q4 (Navigator)

You are given an integer array nums of length n. The neighbours of nums[i] are nums[i - 1] and nums[i + 1] (when they exist). An element is a peak element if it is strictly greater than its neighbours. For convenience, assume nums[-1] = nums[n] = -infinity.

In this question nums strictly increases up to its largest value and then strictly decreases (either part may be empty, so the array may only go up or only go down). This means there is exactly one peak element.

Find the index of the peak element in O(log n) time, so checking every element is not enough.

Input

nums (list of int) — a non-empty array that strictly increases and then strictly decreases

For example, given nums = [1, 2, 3, 1]:

Index 1: value 2 is not greater than its right neighbour 3, so it is not a peak

Index 2: value 3 is greater than both neighbours (2 and 1), so it is the peak

Output

A single integer: the index of the peak element.

For the example above:

2

The element at index 2 has value 3, which is greater than both its neighbours.

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

Given an array A of n integers and a target value K, find two elements of A whose sum is equal to K. The two elements can be the same element, so a pair may use the same index twice (A[i] + A[i]). Once a pair is found, the search can stop.

To make the answer unique, follow these rules:

- Return the pair as a list [i, j] with i <= j.
- If more than one pair adds up to K, return the one with the smallest i. If several pairs share that i, return the one with the smallest j.
- If no pair adds up to K (including when A is empty), return [-1, -1].

Input

A (list of int) — the input array of integers, not necessarily sorted

K (int) — the target sum

For example, given A = [3, 1, 7, 4, 5, 9], K = 8:

Three pairs add up to 8: A[0] + A[4] = 3 + 5, A[1] + A[2] = 1 + 7 and A[3] + A[3] = 4 + 4

The pair starting at index 0 has the smallest first index

Output

A list of two indices [i, j] such that A[i] + A[j] == K, or [-1, -1] if there is no such pair.

For the example above:

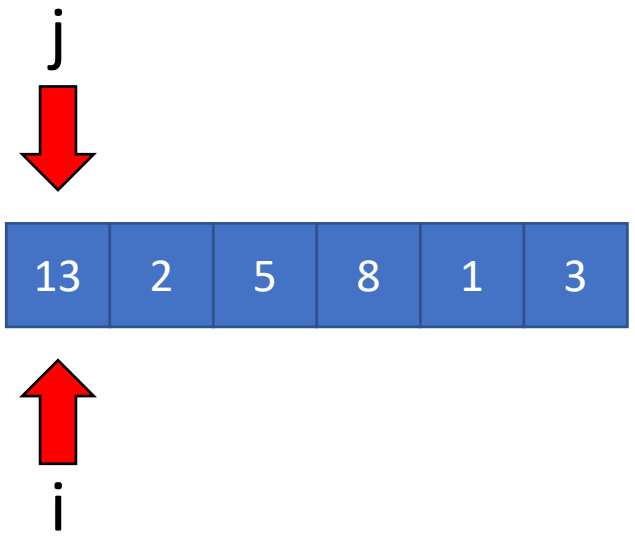
[0, 4]

3 + 5 = 8, and no pair has a first index smaller than 0.

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

```
def dual_search(self, A: List[int], K: int) -> List[int]:
    for i in range(len(A)):
        for j in range(i, len(A)):
            if A[i] + A[j] == K:
                return [i, j]
    return [-1, -1]
```



Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

Given an array A of n integers sorted in ascending order, find two elements of A whose sum is equal to K. The two elements can be the same element, so a pair may use the same index twice (A[i] + A[i]).

Because A is sorted, you do not need to check every pair: your algorithm must run in O(n) time.

To make the answer unique, follow these rules:

- All values in A are different.
- Return the pair as a list [i, j] with i <= j.
- If more than one pair adds up to K, return the one with the smallest i.
- If no pair adds up to K (including when A is empty), return [-1, -1].

Input

A (list of int) — a sorted array of distinct integers

K (int) — the target sum

For example, given A = [1, 3, 4, 5, 7, 9], K = 8:

Three pairs add up to 8: A[0] + A[4] = 1 + 7, A[1] + A[3] = 3 + 5 and A[2] + A[2] = 4 + 4

The pair starting at index 0 has the smallest first index

Output

A list of two indices [i, j] such that A[i] + A[j] == K, or [-1, -1] if there is no such pair.

For the example above:

[0, 4]

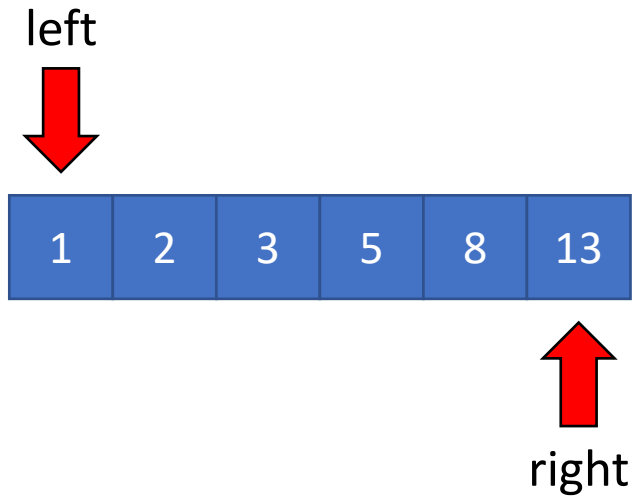
1 + 7 = 8, and no pair has a first index smaller than 0.

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

```
def dual_sorted_search(self, A: List[int], K: int) -> List[int]:
    left, right = 0, len(A) - 1
    while left <= right:
        total = A[left] + A[right]
        if total == K:
            return [left, right]
        elif total < K:
            left += 1
        else:
            right -= 1
    return [-1, -1]
```

K = 12



Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

Given two arrays num1 and num2, each sorted in ascending order, with lengths m and n, find the median of all m + n numbers combined. Your solution must run in O(log(min(m, n))) time, so merging the two arrays is not enough.

The median is the middle value of the combined sorted numbers:

- If m + n is odd, it is the middle element.
- If m + n is even, it is the average of the two middle elements (for example 2.5).

Input

num1 (list of int) — the first sorted array, possibly empty

num2 (list of int) — the second sorted array, possibly empty

At least one of the two arrays is non-empty.

For example, given num1 = [1, 3, 8], num2 = [7, 9, 10, 11]:

Combined and sorted, the numbers are [1, 3, 7, 8, 9, 10, 11]

There are 7 numbers (odd), so the median is the 4th one

Output

A single number: the median of the combined numbers.

For the example above:

8

The 4th of the 7 combined numbers is 8.

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

```python
def find_median_sorted_arrays(self, num1: List[int], num2: List[int]) -> float:
    if len(num1) > len(num2):
        num1, num2 = num2, num1  # binary search on the shorter array

    m, n = len(num1), len(num2)
    low, high = 0, m
    half = (m + n + 1) // 2

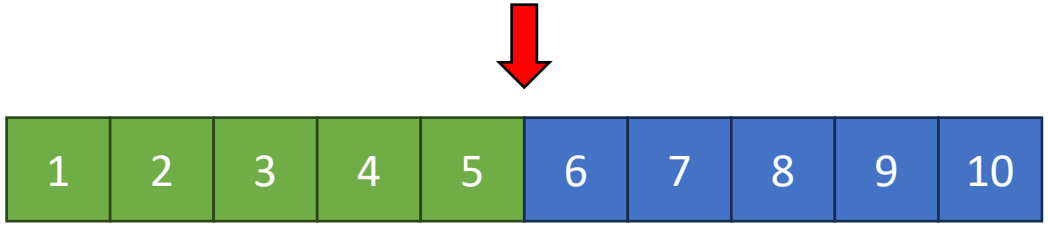
    while low <= high:
        i = (low + high) // 2  # number of elements taken from num1
        j = half - i           # number of elements taken from num2

        left1 = num1[i - 1] if i > 0 else float("-inf")
        right1 = num1[i] if i < m else float("inf")
        left2 = num2[j - 1] if j > 0 else float("-inf")
        right2 = num2[j] if j < n else float("inf")

        if left1 <= right2 and left2 <= right1:
            if (m + n) % 2 == 1:
                return max(left1, left2)
            return (max(left1, left2) + min(right1, right2)) / 2
        elif left1 > right2:
            high = i - 1
        else:
            low = i + 1
```

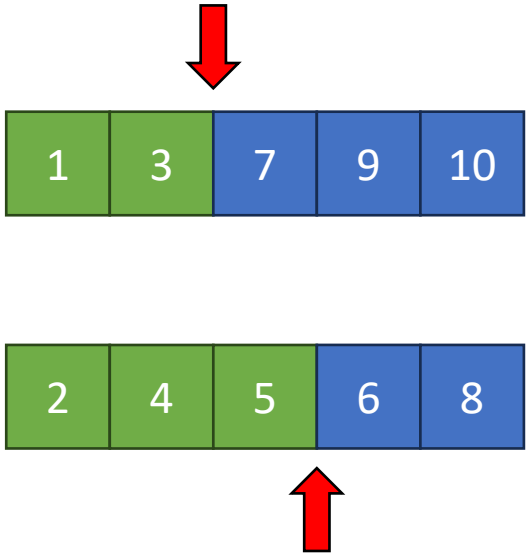
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer



Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

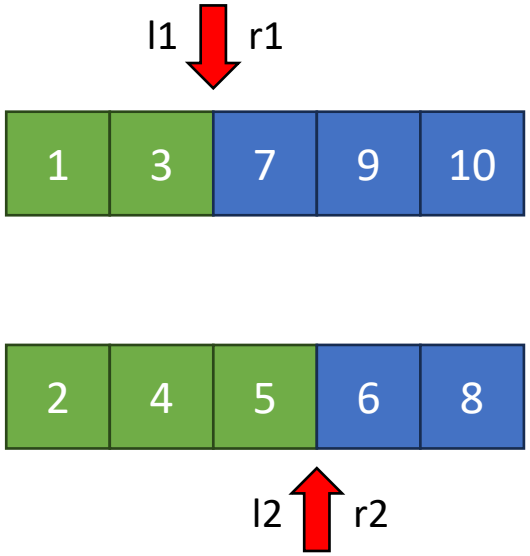


Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

For every element on the left to be smaller than every element on the right

l1 <= r2
l2 <= r1



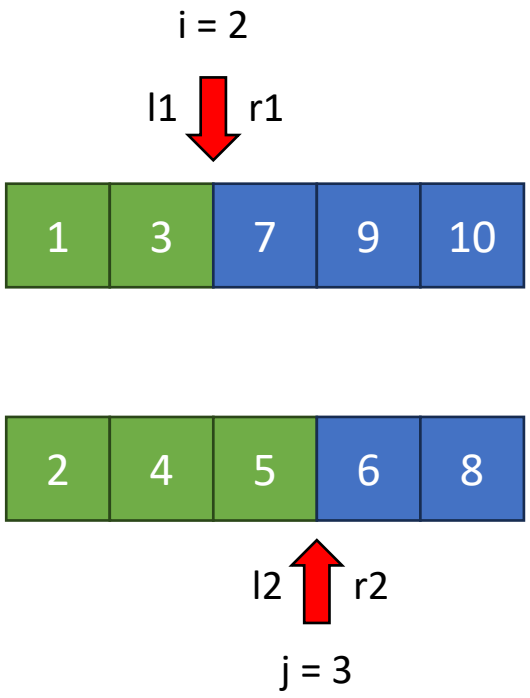
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

For every element on the left to be smaller than every element on the right

$l1 <= r2$

$l2 <= r1$



Number of elements

num1 → i

num2 → j = (m+n+1) // 2 – i

m = len(num1), n = len(num2)

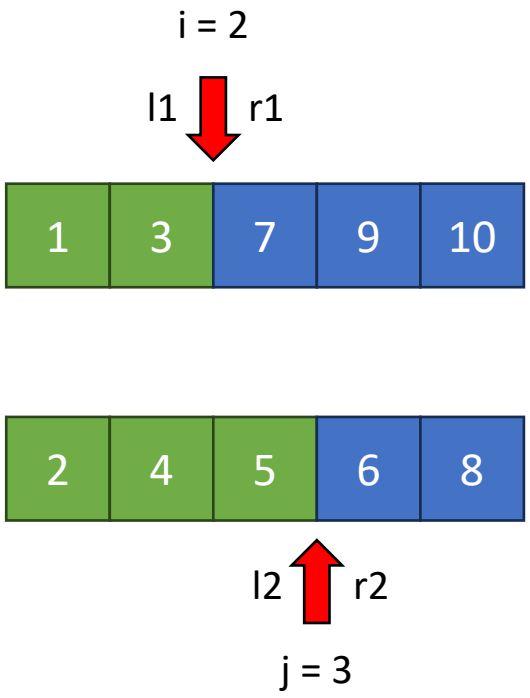
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

For every element on the left to be smaller than every element on the right

$l1 <= r2$

$l2 <= r1$



Number of elements

num1 → i

num2 → j = (m+n+1) // 2 – i

m = len(num1), n = len(num2)

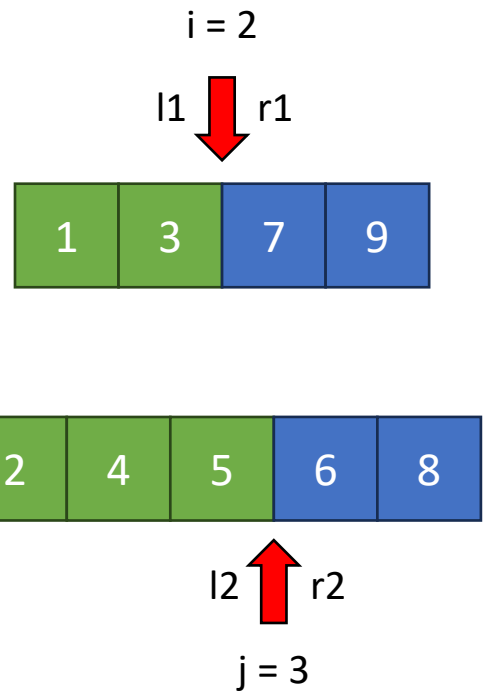
Odd → return max(l1, l2)

Even → return (max(l1, l2) + min(r1, r2)) / 2

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

Odd number of elements



For every element on the left to be smaller than every element on the right

l1 <= r2
l2 <= r1

Number of elements
num1 → i
num2 → j = (m+n+1) // 2 – i

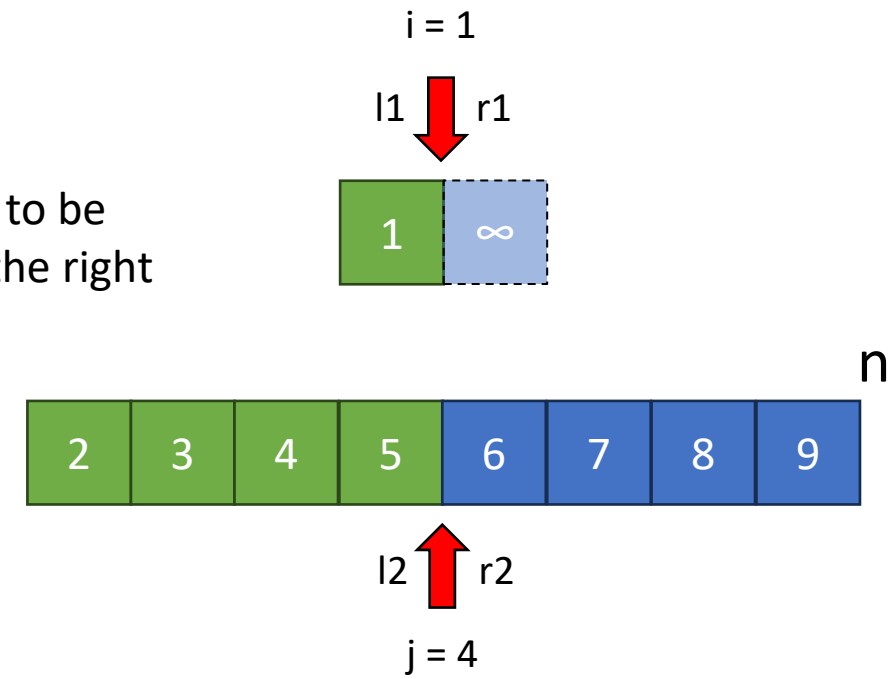
m = len(num1), n = len(num2)

Odd → return max(l1, l2)
Even → return (max(l1, l2) + min(r1, r2)) / 2

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

Unbalanced lists



For every element on the left to be smaller than every element on the right

l1 <= r2
l2 <= r1

Number of elements
num1 → i
num2 → j = (m+n+1) // 2 – i

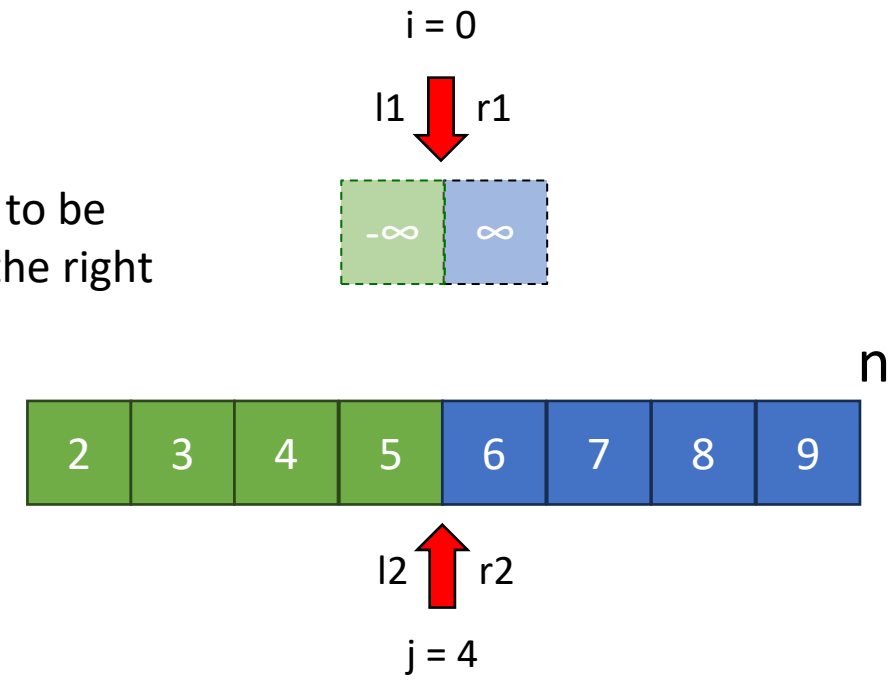
m = len(num1), n = len(num2)

Odd → return max(l1, l2)
Even → return (max(l1, l2) + min(r1, r2)) / 2

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

Single list



For every element on the left to be smaller than every element on the right

l1 <= r2
l2 <= r1

Number of elements
num1 → i
num2 → j = (m+n+1) // 2 – i

m = len(num1), n = len(num2)

Odd → return max(l1, l2)
Even → return (max(l1, l2) + min(r1, r2)) / 2

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

For every element on the left to be smaller than every element on the right

l1 <= r2
l2 <= r1

Number of elements
num1 → i
num2 → j = (m+n+1) // 2 – i

m = len(num1), n = len(num2)

Odd → return max(l1, l2)
Even → return (max(l1, l2)
+ min(r1, r2)) / 2

```python
def find_median_sorted_arrays(self, num1: List[int], num2: List[int]) -> float:
    if len(num1) > len(num2):
        num1, num2 = num2, num1  # binary search on the shorter array

    m, n = len(num1), len(num2)
    low, high = 0, m
    half = (m + n + 1) // 2

    while low <= high:
        i = (low + high) // 2  # number of elements taken from num1
        j = half - i           # number of elements taken from num2

        left1 = num1[i - 1] if i > 0 else float("-inf")
        right1 = num1[i] if i < m else float("inf")
        left2 = num2[j - 1] if j > 0 else float("-inf")
        right2 = num2[j] if j < n else float("inf")

        if left1 <= right2 and left2 <= right1:
            if (m + n) % 2 == 1:
                return max(left1, left2)
            return (max(left1, left2) + min(right1, right2)) / 2
        elif left1 > right2:
            high = i - 1
        else:
            low = i + 1
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

For every element on the left to be smaller than every element on the right

l1 <= r2
l2 <= r1

Number of elements
num1 → i
num2 → j = (m+n+1) // 2 – i

m = len(num1), n = len(num2)

Odd → return max(l1, l2)
Even → return (max(l1, l2)
+ min(r1, r2)) / 2

```
def find_median_sorted_arrays(self, num1: List[int], num2: List[int]) -> float:
    if len(num1) > len(num2):
        num1, num2 = num2, num1  # binary search on the shorter array

    m, n = len(num1), len(num2)
    low, high = 0, m
    half = (m + n + 1) // 2

    while low <= high:
        i = (low + high) // 2  # number of elements taken from num1
        j = half - i           # number of elements taken from num2

        left1 = num1[i - 1] if i > 0 else float("-inf")
        right1 = num1[i] if i < m else float("inf")
        left2 = num2[j - 1] if j > 0 else float("-inf")
        right2 = num2[j] if j < n else float("inf")

        if left1 <= right2 and left2 <= right1:
            if (m + n) % 2 == 1:
                return max(left1, left2)
            return (max(left1, left2) + min(right1, right2)) / 2
        elif left1 > right2:
            high = i - 1
        else:
            low = i + 1
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

For every element on the left to be smaller than every element on the right

l1 <= r2

l2 <= r1

Number of elements

num1 → i

num2 → j = (m+n+1) // 2 – i

m = len(num1), n = len(num2)

Odd → return max(l1, l2)

Even → return (max(l1, l2) + min(r1, r2)) / 2

```python
def find_median_sorted_arrays(self, num1: List[int], num2: List[int]) -> float:
    if len(num1) > len(num2):
        num1, num2 = num2, num1  # binary search on the shorter array

    m, n = len(num1), len(num2)
    low, high = 0, m
    half = (m + n + 1) // 2

    while low <= high:
        i = (low + high) // 2  # number of elements taken from num1
        j = half - i           # number of elements taken from num2

        left1 = num1[i - 1] if i > 0 else float("-inf")
        right1 = num1[i] if i < m else float("inf")
        left2 = num2[j - 1] if j > 0 else float("-inf")
        right2 = num2[j] if j < n else float("inf")

        if left1 <= right2 and left2 <= right1:
            if (m + n) % 2 == 1:
                return max(left1, left2)
            return (max(left1, left2) + min(right1, right2)) / 2
        elif left1 > right2:
            high = i - 1
        else:
            low = i + 1
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

For every element on the left to be smaller than every element on the right

l1 <= r2
l2 <= r1

Number of elements
num1 → i
num2 → j = (m+n+1) // 2 – i

m = len(num1), n = len(num2)

Odd → return max(l1, l2)
Even → return (max(l1, l2)
+ min(r1, r2)) / 2

```
def find_median_sorted_arrays(self, num1: List[int], num2: List[int]) -> float:
    if len(num1) > len(num2):
        num1, num2 = num2, num1  # binary search on the shorter array

    m, n = len(num1), len(num2)
    low, high = 0, m
    half = (m + n + 1) // 2

    while low <= high:
        i = (low + high) // 2  # number of elements taken from num1
        j = half - i           # number of elements taken from num2

        left1 = num1[i - 1] if i > 0 else float("-inf")
        right1 = num1[i] if i < m else float("inf")
        left2 = num2[j - 1] if j > 0 else float("-inf")
        right2 = num2[j] if j < n else float("inf")

        if left1 <= right2 and left2 <= right1:
            if (m + n) % 2 == 1:
                return max(left1, left2)
            return (max(left1, left2) + min(right1, right2)) / 2
        elif left1 > right2:
            high = i - 1
        else:
            low = i + 1
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

For every element on the left to be smaller than every element on the right

l1 <= r2

l2 <= r1

Number of elements

num1 → i

num2 → j = (m+n+1) // 2 – i

m = len(num1), n = len(num2)

Odd → return max(l1, l2)

Even → return (max(l1, l2) + min(r1, r2)) / 2

```python
def find_median_sorted_arrays(self, num1: List[int], num2: List[int]) -> float:
    if len(num1) > len(num2):
        num1, num2 = num2, num1  # binary search on the shorter array

    m, n = len(num1), len(num2)
    low, high = 0, m
    half = (m + n + 1) // 2

    while low <= high:
        i = (low + high) // 2  # number of elements taken from num1
        j = half - i           # number of elements taken from num2

        left1 = num1[i - 1] if i > 0 else float("-inf")
        right1 = num1[i] if i < m else float("inf")
        left2 = num2[j - 1] if j > 0 else float("-inf")
        right2 = num2[j] if j < n else float("inf")

        if left1 <= right2 and left2 <= right1:
            if (m + n) % 2 == 1:
                return max(left1, left2)
            return (max(left1, left2) + min(right1, right2)) / 2
        elif left1 > right2:
            high = i - 1
        else:
            low = i + 1
```

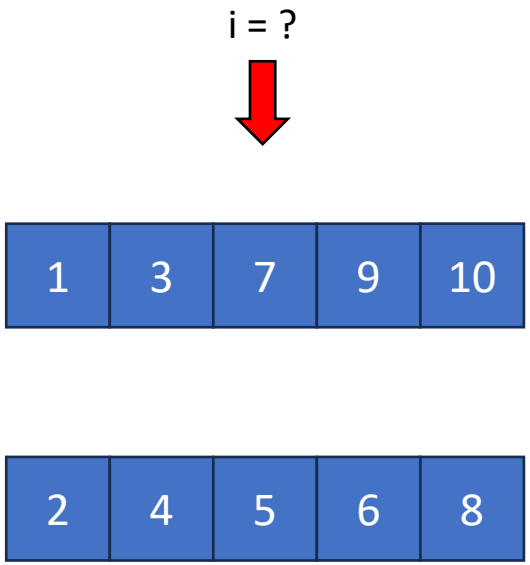
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

For every element on the left to be smaller than every element on the right

$l1 <= r2$

$l2 <= r1$



Number of elements

num1 → i

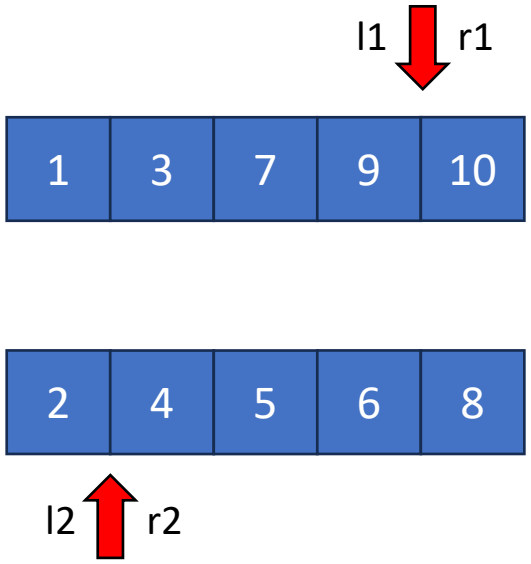
num2 → j = (m+n+1)/2 – i

m = len(num1), n = len(num2)

Content Copyright Nanyang Technological University

Driver - Answer

Too far to the right



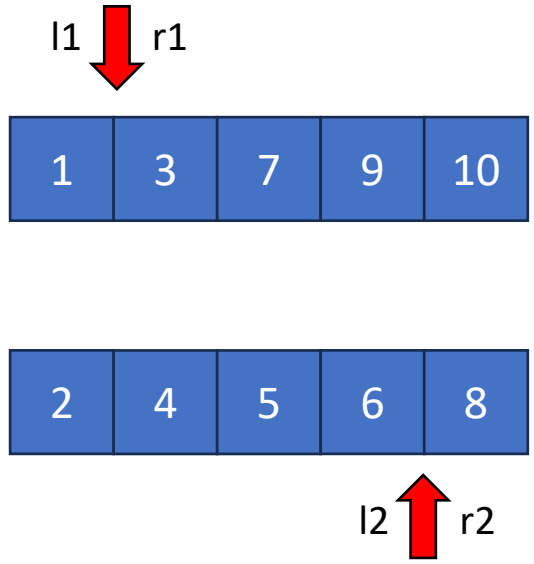
l1 > r2

For every element on the left to be smaller than every element on the right

l1 <= r2

l2 <= r1

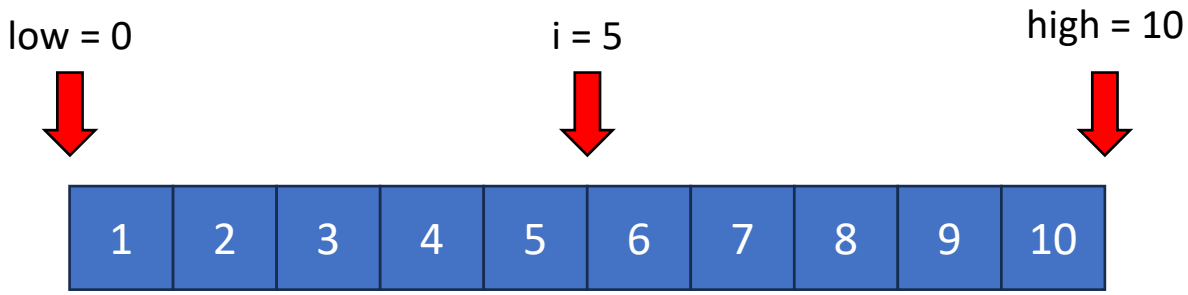
Too far to the left



l2 > r1

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer



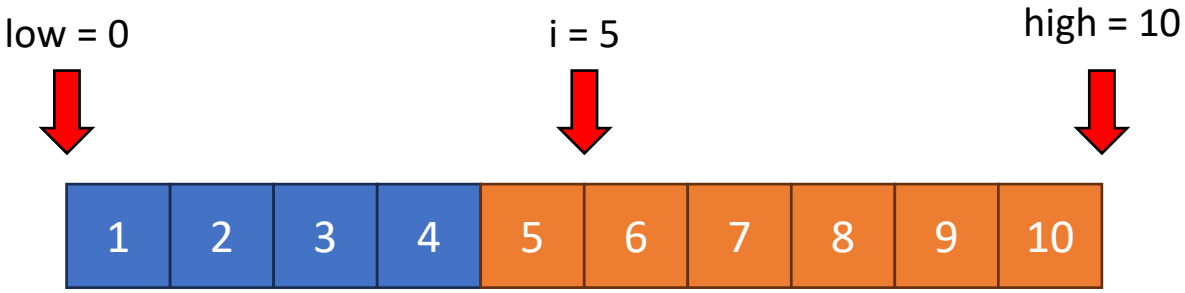
i = (low + high) / 2

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

Move to the left

$i < 5$



$i = (low + high) / 2$

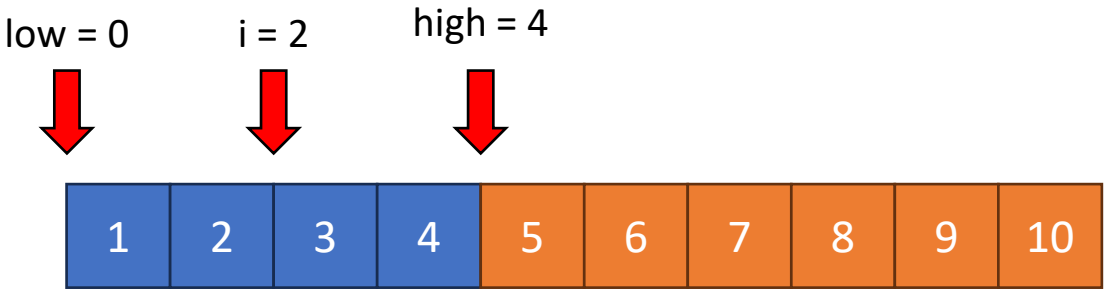
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

Move to the left

i < 5

high = i - 1



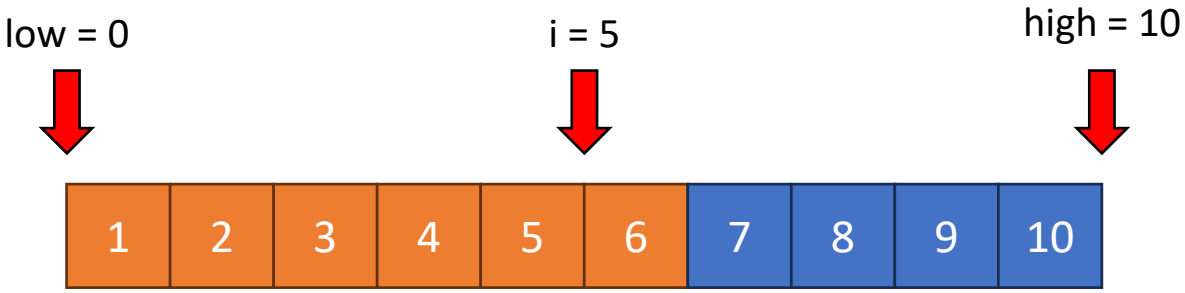
i = (low + high) / 2

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

Move to the right

i > 5



i = (low + high) / 2

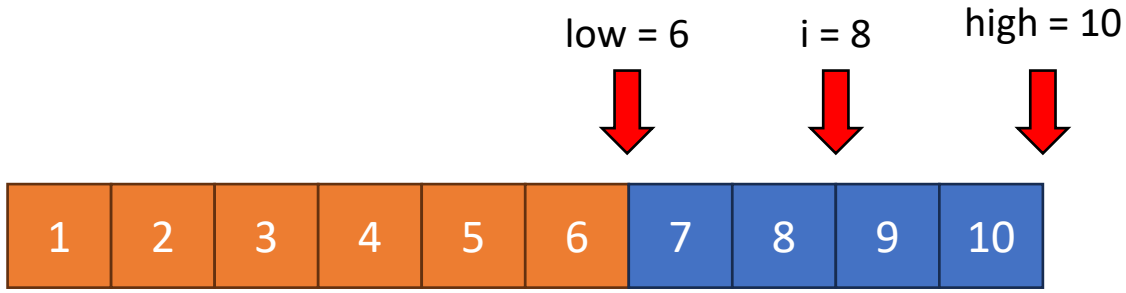
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

Move to the right

i > 5

low = i + 1



i = (low + high) / 2

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

Too far to the right

l1 > r2

Too far to the left

l2 > r1

i = (low + high) / 2

Move to the left

high = i - 1

Move to the right

low = i + 1

```python
def find_median_sorted_arrays(self, num1: List[int], num2: List[int]) -> float:
    if len(num1) > len(num2):
        num1, num2 = num2, num1  # binary search on the shorter array

    m, n = len(num1), len(num2)
    low, high = 0, m
    half = (m + n + 1) // 2

    while low <= high:
        i = (low + high) // 2  # number of elements taken from num1
        j = half - i           # number of elements taken from num2

        left1 = num1[i - 1] if i > 0 else float("-inf")
        right1 = num1[i] if i < m else float("inf")
        left2 = num2[j - 1] if j > 0 else float("-inf")
        right2 = num2[j] if j < n else float("inf")

        if left1 <= right2 and left2 <= right1:
            if (m + n) % 2 == 1:
                return max(left1, left2)
            return (max(left1, left2) + min(right1, right2)) / 2
        elif left1 > right2:
            high = i - 1
        else:
            low = i + 1
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

Too far to the right

l1 > r2

Too far to the left

l2 > r1

i = (low + high) / 2

Move to the left

high = i - 1

Move to the right

low = i + 1

```
def find_median_sorted_arrays(self, num1: List[int], num2: List[int]) -> float:
    if len(num1) > len(num2):
        num1, num2 = num2, num1  # binary search on the shorter array

    m, n = len(num1), len(num2)
    low, high = 0, m
    half = (m + n + 1) // 2

    while low <= high:
        i = (low + high) // 2  # number of elements taken from num1
        j = half - i           # number of elements taken from num2

        left1 = num1[i - 1] if i > 0 else float("-inf")
        right1 = num1[i] if i < m else float("inf")
        left2 = num2[j - 1] if j > 0 else float("-inf")
        right2 = num2[j] if j < n else float("inf")

        if left1 <= right2 and left2 <= right1:
            if (m + n) % 2 == 1:
                return max(left1, left2)
            return (max(left1, left2) + min(right1, right2)) / 2
        elif left1 > right2:
            high = i - 1
        else:
            low = i + 1
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

Too far to the right

l1 > r2

Too far to the left

l2 > r1

i = (low + high) / 2

Move to the left

high = i - 1

Move to the right

low = i + 1

```python
def find_median_sorted_arrays(self, num1: List[int], num2: List[int]) -> float:
    if len(num1) > len(num2):
        num1, num2 = num2, num1  # binary search on the shorter array

    m, n = len(num1), len(num2)
    low, high = 0, m
    half = (m + n + 1) // 2

    while low <= high:
        i = (low + high) // 2  # number of elements taken from num1
        j = half - i           # number of elements taken from num2

        left1 = num1[i - 1] if i > 0 else float("-inf")
        right1 = num1[i] if i < m else float("inf")
        left2 = num2[j - 1] if j > 0 else float("-inf")
        right2 = num2[j] if j < n else float("inf")

        if left1 <= right2 and left2 <= right1:
            if (m + n) % 2 == 1:
                return max(left1, left2)
            return (max(left1, left2) + min(right1, right2)) / 2
        elif left1 > right2:
            high = i - 1
        else:
            low = i + 1
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

Too far to the right

l1 > r2

Too far to the left

l2 > r1

i = (low + high) / 2

Move to the left

high = i - 1

Move to the right

low = i + 1

```
def find_median_sorted_arrays(self, num1: List[int], num2: List[int]) -> float:
    if len(num1) > len(num2):
        num1, num2 = num2, num1  # binary search on the shorter array

    m, n = len(num1), len(num2)
    low, high = 0, m
    half = (m + n + 1) // 2

    while low <= high:
        i = (low + high) // 2  # number of elements taken from num1
        j = half - i           # number of elements taken from num2

        left1 = num1[i - 1] if i > 0 else float("-inf")
        right1 = num1[i] if i < m else float("inf")
        left2 = num2[j - 1] if j > 0 else float("-inf")
        right2 = num2[j] if j < n else float("inf")

        if left1 <= right2 and left2 <= right1:
            if (m + n) % 2 == 1:
                return max(left1, left2)
            return (max(left1, left2) + min(right1, right2)) / 2
        elif left1 > right2:
            high = i - 1
        else:
            low = i + 1
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

Too far to the right

l1 > r2

Too far to the left

l2 > r1

i = (low + high) / 2

Move to the left

high = i - 1

Move to the right

low = i + 1

```python
def find_median_sorted_arrays(self, num1: List[int], num2: List[int]) -> float:
    if len(num1) > len(num2):
        num1, num2 = num2, num1  # binary search on the shorter array

    m, n = len(num1), len(num2)
    low, high = 0, m
    half = (m + n + 1) // 2

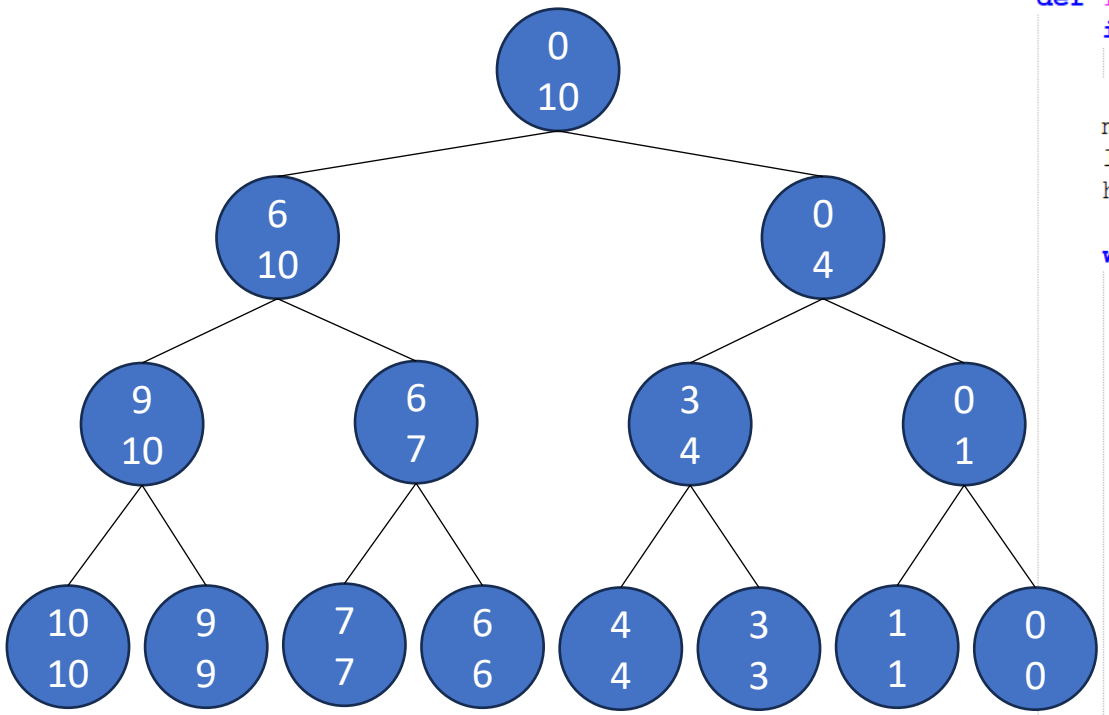
    while low <= high:
        i = (low + high) // 2  # number of elements taken from num1
        j = half - i           # number of elements taken from num2

        left1 = num1[i - 1] if i > 0 else float("-inf")
        right1 = num1[i] if i < m else float("inf")
        left2 = num2[j - 1] if j > 0 else float("-inf")
        right2 = num2[j] if j < n else float("inf")

        if left1 <= right2 and left2 <= right1:
            if (m + n) % 2 == 1:
                return max(left1, left2)
            return (max(left1, left2) + min(right1, right2)) / 2
        elif left1 > right2:
            high = i - 1
        else:
            low = i + 1
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer




```python
def find_median_sorted_arrays(self, num1: List[int], num2: List[int]) -> float:
    if len(num1) > len(num2):
        num1, num2 = num2, num1  # binary search on the shorter array

    m, n = len(num1), len(num2)
    low, high = 0, m
    half = (m + n + 1) // 2

    while low <= high:
        i = (low + high) // 2  # number of elements taken from num1
        j = half - i           # number of elements taken from num2

        left1 = num1[i - 1] if i > 0 else float("-inf")
        right1 = num1[i] if i < m else float("inf")
        left2 = num2[j - 1] if j > 0 else float("-inf")
        right2 = num2[j] if j < n else float("inf")

        if left1 <= right2 and left2 <= right1:
            if (m + n) % 2 == 1:
                return max(left1, left2)
            return (max(left1, left2) + min(right1, right2)) / 2
        elif left1 > right2:
            high = i - 1
        else:
            low = i + 1
```

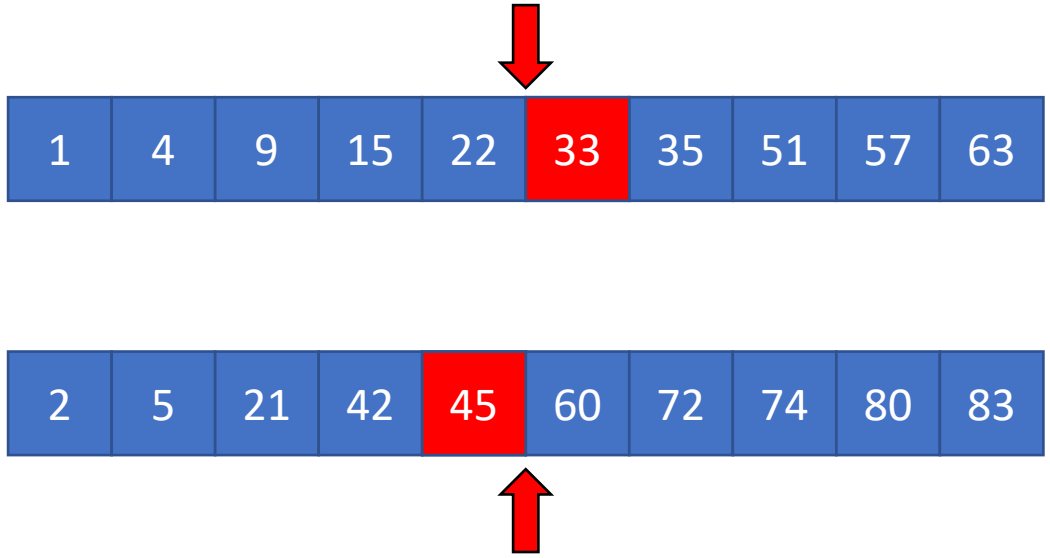
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer (Extra)

*Extra alternate solution from last semester for reference

imin = 0 imax = 10

i = 5 j = 5




```
while imin <= imax:
    i = (imin + imax) // 2
    j = half_len - i

    if i < m and num1[i] < num2[j - 1]:
        imin = i + 1  # i is too small, move right
    elif i > 0 and num1[i - 1] > num2[j]:
        imax = i - 1  # i is too big, move left
    else:
        max_of_left = 0
        if i == 0:
            max_of_left = num2[j - 1]
        elif j == 0:
            max_of_left = num1[i - 1]
        else:
            max_of_left = max(num1[i - 1], num2[j - 1])

        if (m + n) % 2 == 1:
            return max_of_left  # Odd total length

        min_of_right = 0
        if i == m:
            min_of_right = num2[j]
        elif j == n:
            min_of_right = num1[i]
        else:
            min_of_right = min(num1[i], num2[j])

        return (max_of_left + min_of_right) / 2.0  # Even total length
```

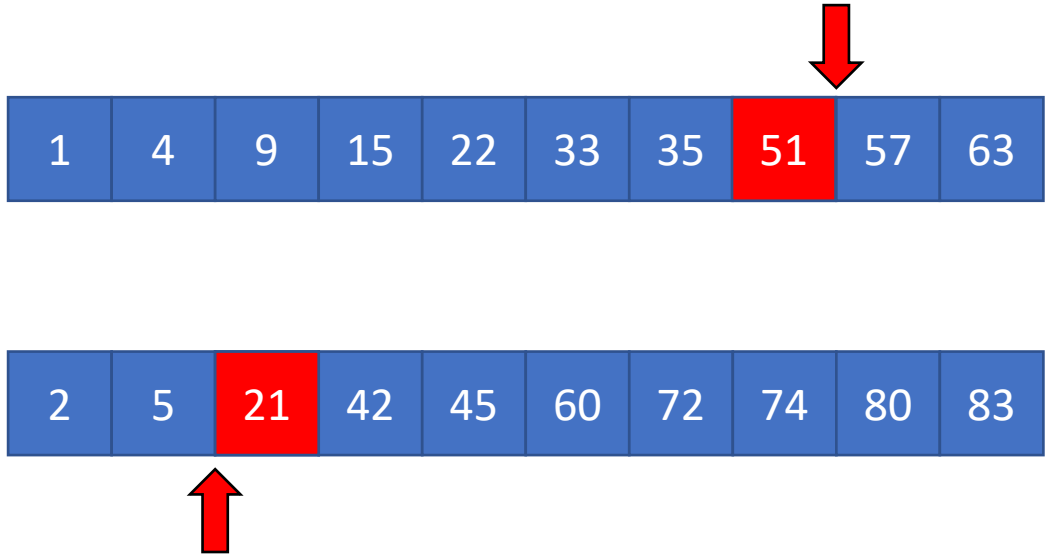
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer (Extra)

*Extra alternate solution from last semester for reference

imin = 6 imax = 10

i = 8 j = 2




```
while imin <= imax:
    i = (imin + imax) // 2
    j = half_len - i

    if i < m and num1[i] < num2[j - 1]:
        imin = i + 1  # i is too small, move right
    elif i > 0 and num1[i - 1] > num2[j]:
        imax = i - 1  # i is too big, move left
    else:
        max_of_left = 0
        if i == 0:
            max_of_left = num2[j - 1]
        elif j == 0:
            max_of_left = num1[i - 1]
        else:
            max_of_left = max(num1[i - 1], num2[j - 1])

        if (m + n) % 2 == 1:
            return max_of_left  # Odd total length

        min_of_right = 0
        if i == m:
            min_of_right = num2[j]
        elif j == n:
            min_of_right = num1[i]
        else:
            min_of_right = min(num1[i], num2[j])

        return (max_of_left + min_of_right) / 2.0  # Even total length
```

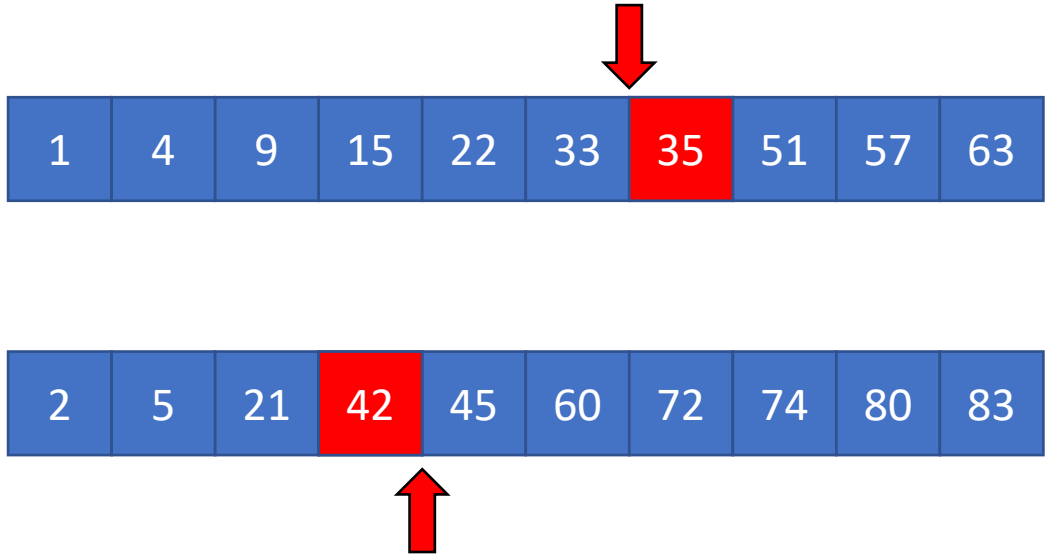
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer (Extra)

*Extra alternate solution from last semester for reference

imin = 6 imax = 7

i = 6 j = 4




```
while imin <= imax:
    i = (imin + imax) // 2
    j = half_len - i

    if i < m and num1[i] < num2[j - 1]:
        imin = i + 1  # i is too small, move right
    elif i > 0 and num1[i - 1] > num2[j]:
        imax = i - 1  # i is too big, move left
    else:
        max_of_left = 0
        if i == 0:
            max_of_left = num2[j - 1]
        elif j == 0:
            max_of_left = num1[i - 1]
        else:
            max_of_left = max(num1[i - 1], num2[j - 1])

        if (m + n) % 2 == 1:
            return max_of_left  # Odd total length

        min_of_right = 0
        if i == m:
            min_of_right = num2[j]
        elif j == n:
            min_of_right = num1[i]
        else:
            min_of_right = min(num1[i], num2[j])

        return (max_of_left + min_of_right) / 2.0  # Even total length
```

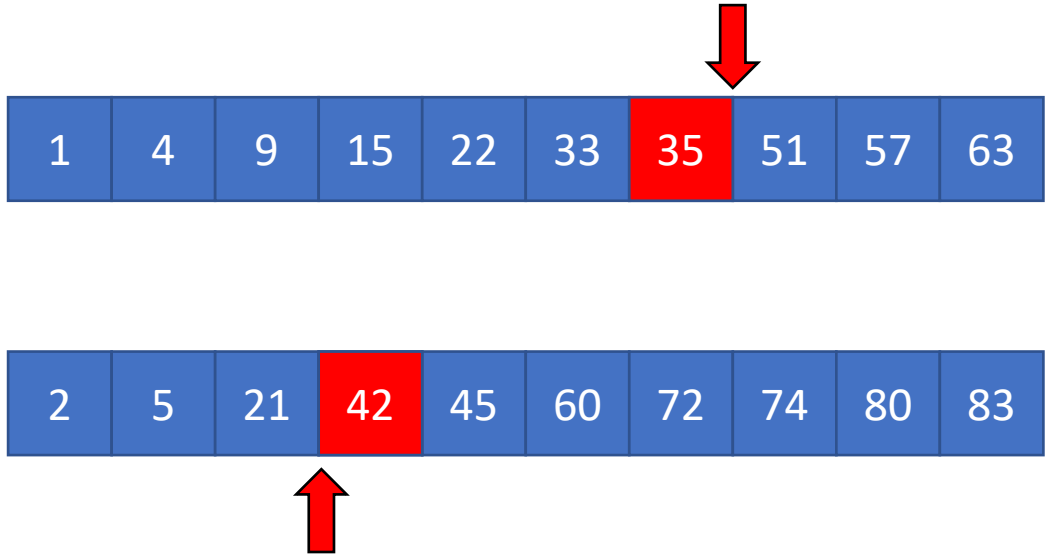
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer (Extra)

*Extra alternate solution from last semester for reference

imin = 7 imax = 7

i = 7 j = 3



max_of_left = 35

min_of_right = 42

```
while imin <= imax:
    i = (imin + imax) // 2
    j = half_len - i

    if i < m and num1[i] < num2[j - 1]:
        imin = i + 1  # i is too small, move right
    elif i > 0 and num1[i - 1] > num2[j]:
        imax = i - 1  # i is too big, move left
    else:
        max_of_left = 0
        if i == 0:
            max_of_left = num2[j - 1]
        elif j == 0:
            max_of_left = num1[i - 1]
        else:
            max_of_left = max(num1[i - 1], num2[j - 1])

        if (m + n) % 2 == 1:
            return max_of_left  # Odd total length

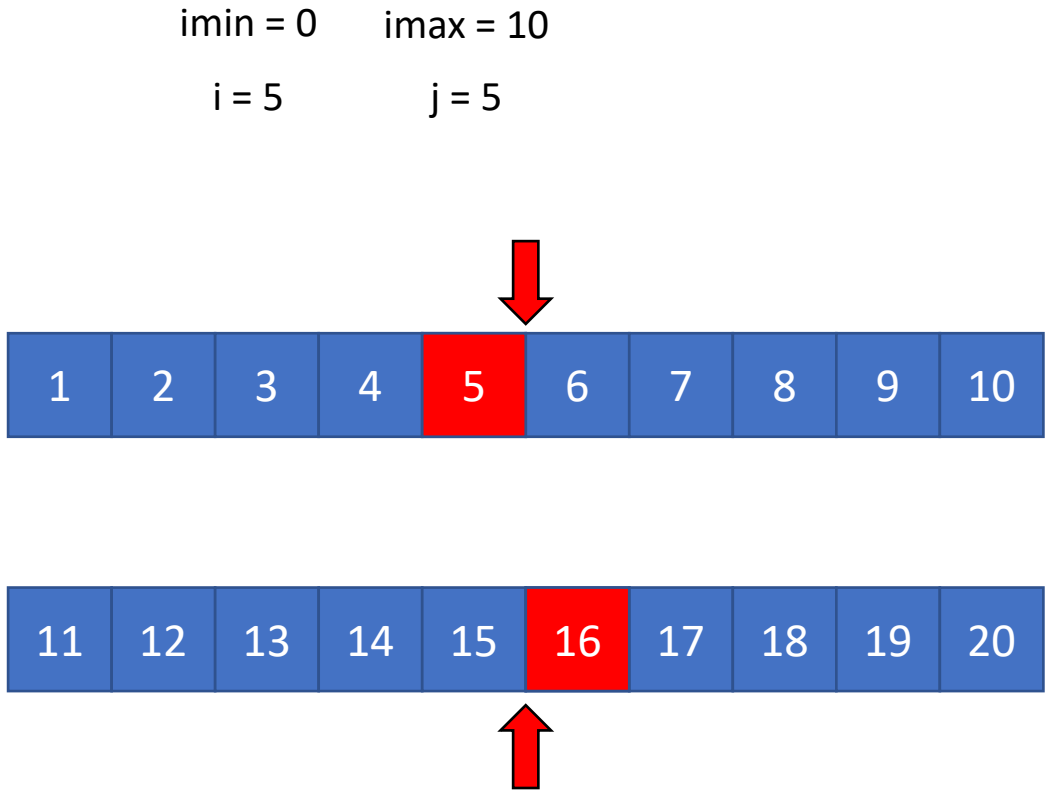
        min_of_right = 0
        if i == m:
            min_of_right = num2[j]
        elif j == n:
            min_of_right = num1[i]
        else:
            min_of_right = min(num1[i], num2[j])

        return (max_of_left + min_of_right) / 2.0  # Even total length
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer (Extra)

*Extra alternate solution from last semester for reference




```python
while imin <= imax:
    i = (imin + imax) // 2
    j = half_len - i

    if i < m and num1[i] < num2[j - 1]:
        imin = i + 1  # i is too small, move right
    elif i > 0 and num1[i - 1] > num2[j]:
        imax = i - 1  # i is too big, move left
    else:
        max_of_left = 0
        if i == 0:
            max_of_left = num2[j - 1]
        elif j == 0:
            max_of_left = num1[i - 1]
        else:
            max_of_left = max(num1[i - 1], num2[j - 1])

        if (m + n) % 2 == 1:
            return max_of_left  # Odd total length

        min_of_right = 0
        if i == m:
            min_of_right = num2[j]
        elif j == n:
            min_of_right = num1[i]
        else:
            min_of_right = min(num1[i], num2[j])

        return (max_of_left + min_of_right) / 2.0  # Even total length
```

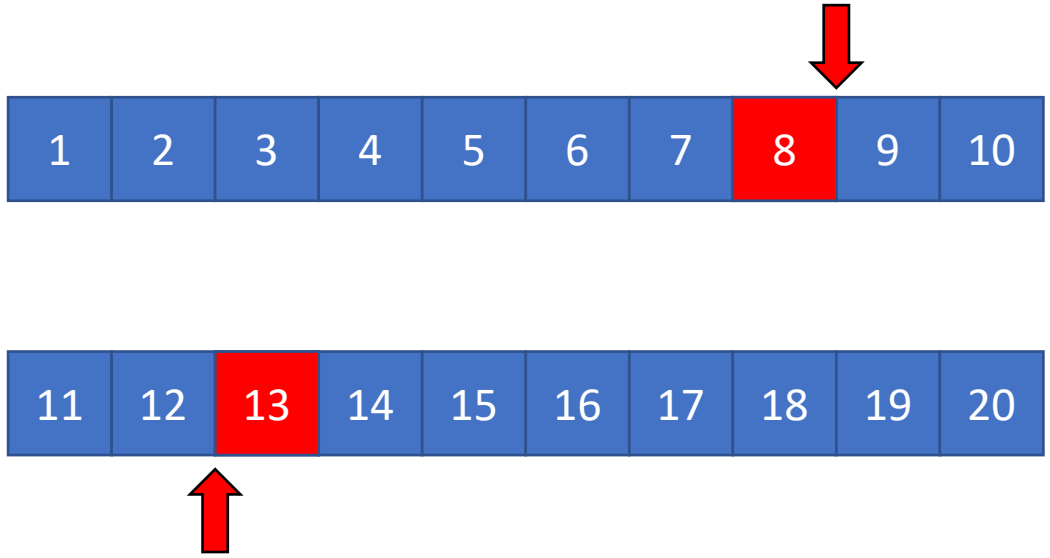
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer (Extra)

*Extra alternate solution from last semester for reference

imin = 6 imax = 10

i = 8 j = 2




```
while imin <= imax:
    i = (imin + imax) // 2
    j = half_len - i

    if i < m and num1[i] < num2[j - 1]:
        imin = i + 1  # i is too small, move right
    elif i > 0 and num1[i - 1] > num2[j]:
        imax = i - 1  # i is too big, move left
    else:
        max_of_left = 0
        if i == 0:
            max_of_left = num2[j - 1]
        elif j == 0:
            max_of_left = num1[i - 1]
        else:
            max_of_left = max(num1[i - 1], num2[j - 1])

        if (m + n) % 2 == 1:
            return max_of_left  # Odd total length

        min_of_right = 0
        if i == m:
            min_of_right = num2[j]
        elif j == n:
            min_of_right = num1[i]
        else:
            min_of_right = min(num1[i], num2[j])

        return (max_of_left + min_of_right) / 2.0  # Even total length
```

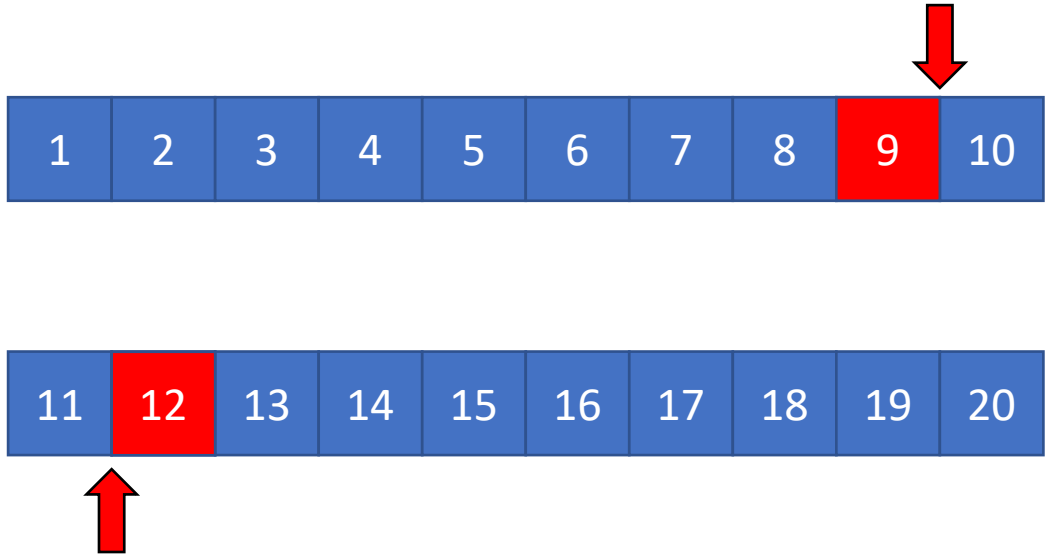
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer (Extra)

*Extra alternate solution from last semester for reference

imin = 9 imax = 10

i = 9 j = 1




```
while imin <= imax:
    i = (imin + imax) // 2
    j = half_len - i

    if i < m and num1[i] < num2[j - 1]:
        imin = i + 1  # i is too small, move right
    elif i > 0 and num1[i - 1] > num2[j]:
        imax = i - 1  # i is too big, move left
    else:
        max_of_left = 0
        if i == 0:
            max_of_left = num2[j - 1]
        elif j == 0:
            max_of_left = num1[i - 1]
        else:
            max_of_left = max(num1[i - 1], num2[j - 1])

        if (m + n) % 2 == 1:
            return max_of_left  # Odd total length

        min_of_right = 0
        if i == m:
            min_of_right = num2[j]
        elif j == n:
            min_of_right = num1[i]
        else:
            min_of_right = min(num1[i], num2[j])

        return (max_of_left + min_of_right) / 2.0  # Even total length
```

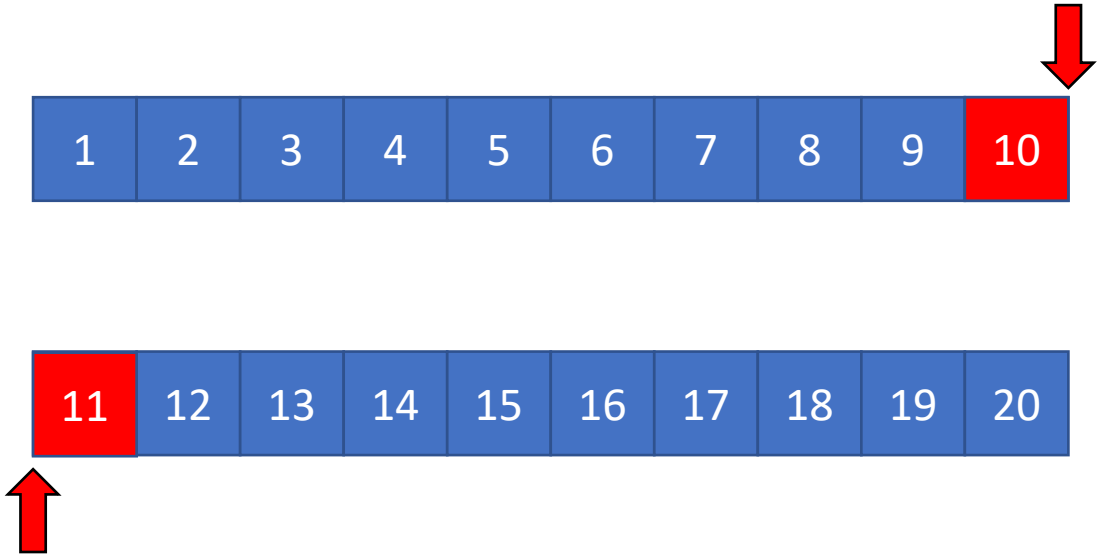
Content Copyright Nanyang Technological University

Q3 (Driver) – Answer (Extra)

*Extra alternate solution from last semester for reference

imin = 10 imax = 10

i = 10 j = 0



max_of_left = 10

min_of_right = 11

```
while imin <= imax:
    i = (imin + imax) // 2
    j = half_len - i

    if i < m and num1[i] < num2[j - 1]:
        imin = i + 1  # i is too small, move right
    elif i > 0 and num1[i - 1] > num2[j]:
        imax = i - 1  # i is too big, move left
    else:
        max_of_left = 0
        if i == 0:
            max_of_left = num2[j - 1]
        elif j == 0:
            max_of_left = num1[i - 1]
        else:
            max_of_left = max(num1[i - 1], num2[j - 1])

        if (m + n) % 2 == 1:
            return max_of_left  # Odd total length

        min_of_right = 0
        if i == m:
            min_of_right = num2[j]
        elif j == n:
            min_of_right = num1[i]
        else:
            min_of_right = min(num1[i], num2[j])

        return (max_of_left + min_of_right) / 2.0  # Even total length
```

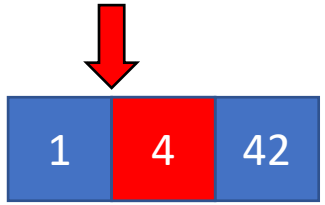
Content Copyright Nanyang Technological University

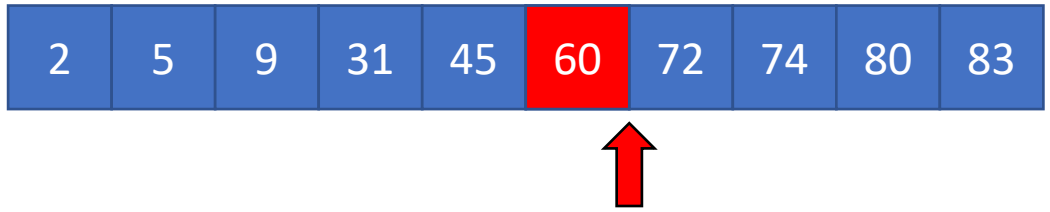
Q3 (Driver) – Answer (Extra)

*Extra alternate solution from last semester for reference

imin = 0 imax = 3

i = 1 j = 6






```
while imin <= imax:
    i = (imin + imax) // 2
    j = half_len - i

    if i < m and num1[i] < num2[j - 1]:
        imin = i + 1  # i is too small, move right
    elif i > 0 and num1[i - 1] > num2[j]:
        imax = i - 1  # i is too big, move left
    else:
        max_of_left = 0
        if i == 0:
            max_of_left = num2[j - 1]
        elif j == 0:
            max_of_left = num1[i - 1]
        else:
            max_of_left = max(num1[i - 1], num2[j - 1])

        if (m + n) % 2 == 1:
            return max_of_left  # Odd total length

        min_of_right = 0
        if i == m:
            min_of_right = num2[j]
        elif j == n:
            min_of_right = num1[i]
        else:
            min_of_right = min(num1[i], num2[j])

        return (max_of_left + min_of_right) / 2.0  # Even total length
```

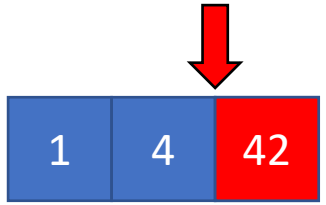
Content Copyright Nanyang Technological University

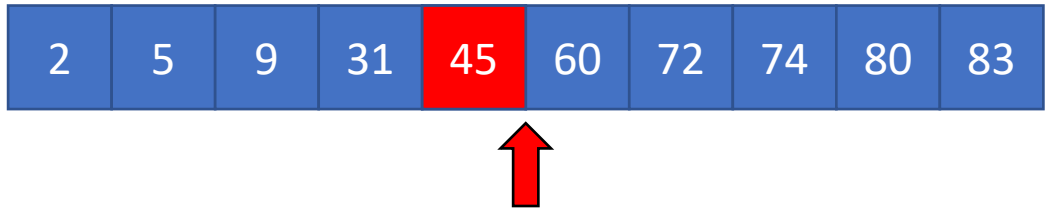
Q3 (Driver) – Answer (Extra)

*Extra alternate solution from last semester for reference

imin = 2 imax = 3

i = 2 j = 5






```
while imin <= imax:
    i = (imin + imax) // 2
    j = half_len - i

    if i < m and num1[i] < num2[j - 1]:
        imin = i + 1  # i is too small, move right
    elif i > 0 and num1[i - 1] > num2[j]:
        imax = i - 1  # i is too big, move left
    else:
        max_of_left = 0
        if i == 0:
            max_of_left = num2[j - 1]
        elif j == 0:
            max_of_left = num1[i - 1]
        else:
            max_of_left = max(num1[i - 1], num2[j - 1])

        if (m + n) % 2 == 1:
            return max_of_left  # Odd total length

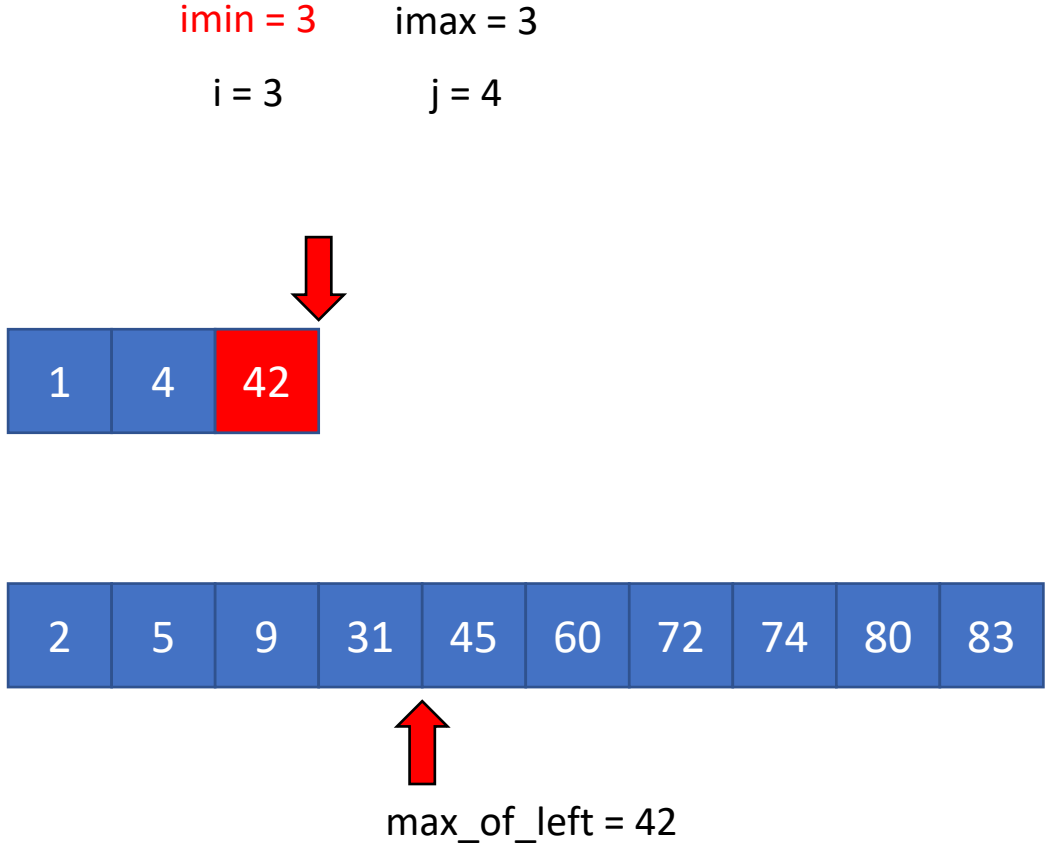
        min_of_right = 0
        if i == m:
            min_of_right = num2[j]
        elif j == n:
            min_of_right = num1[i]
        else:
            min_of_right = min(num1[i], num2[j])

        return (max_of_left + min_of_right) / 2.0  # Even total length
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer (Extra)

*Extra alternate solution from last semester for reference




```python
while imin <= imax:
    i = (imin + imax) // 2
    j = half_len - i

    if i < m and num1[i] < num2[j - 1]:
        imin = i + 1  # i is too small, move right
    elif i > 0 and num1[i - 1] > num2[j]:
        imax = i - 1  # i is too big, move left
    else:
        max_of_left = 0
        if i == 0:
            max_of_left = num2[j - 1]
        elif j == 0:
            max_of_left = num1[i - 1]
        else:
            max_of_left = max(num1[i - 1], num2[j - 1])

        if (m + n) % 2 == 1:
            return max_of_left  # Odd total length

        min_of_right = 0
        if i == m:
            min_of_right = num2[j]
        elif j == n:
            min_of_right = num1[i]
        else:
            min_of_right = min(num1[i], num2[j])

        return (max_of_left + min_of_right) / 2.0  # Even total length
```

Content Copyright Nanyang Technological University

Q4 (Navigator) – Answer

You are given an integer array nums of length n. The neighbours of nums[i] are nums[i - 1] and nums[i + 1] (when they exist). An element is a peak element if it is strictly greater than its neighbours. For convenience, assume nums[-1] = nums[n] = -infinity.

In this question nums strictly increases up to its largest value and then strictly decreases (either part may be empty, so the array may only go up or only go down). This means there is exactly one peak element.

Find the index of the peak element in O(log n) time, so checking every element is not enough.

Input

nums (list of int) — a non-empty array that strictly increases and then strictly decreases

For example, given nums = [1, 2, 3, 1]:

Index 1: value 2 is not greater than its right neighbour 3, so it is not a peak

Index 2: value 3 is greater than both neighbours (2 and 1), so it is the peak

Output

A single integer: the index of the peak element.

For the example above:

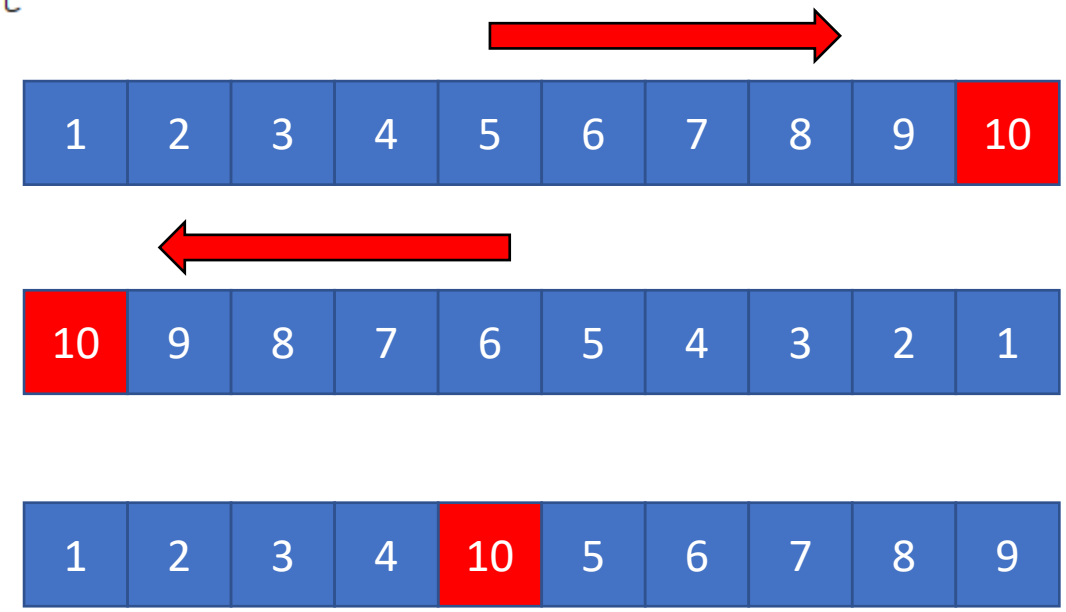
2

The element at index 2 has value 3, which is greater than both its neighbours.

Content Copyright Nanyang Technological University

Q4 (Navigator) – Answer

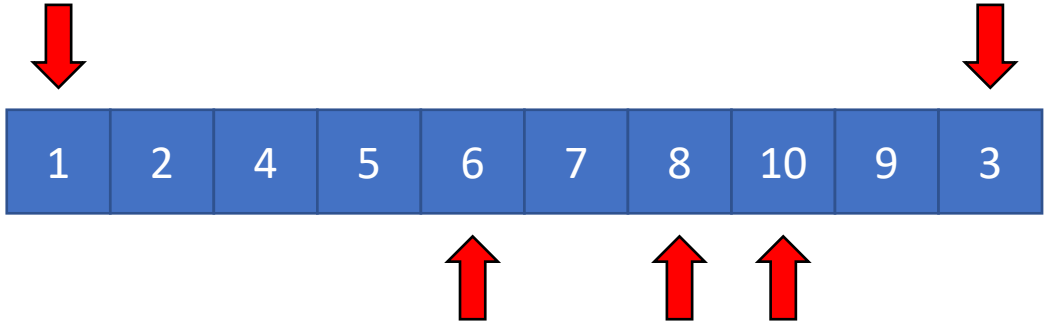
```python
def find_peak(self, nums: List[int]) -> int:
    left, right = 0, len(nums) - 1
    while left < right:
        mid = (left + right) // 2
        if nums[mid] < nums[mid + 1]:
            left = mid + 1  # still going up: the peak is to the right of mid
        else:
            right = mid     # going down: the peak is mid or to its left
    return left
```



Content Copyright Nanyang Technological University

Q4 (Navigator) – Answer

```python
def find_peak(self, nums: List[int]) -> int:
    left, right = 0, len(nums) - 1
    while left < right:
        mid = (left + right) // 2
        if nums[mid] < nums[mid + 1]:
            left = mid + 1  # still going up: the peak is to the right of mid
        else:
            right = mid     # going down: the peak is mid or to its left
    return left
```



Content Copyright Nanyang Technological University

Q4 (Navigator) – Answer

create two variables as the first and last index

while left is less than right, find the middle index

if the middle number is less than the next number, update the left index, otherwise update the right index

return left

```
def find_peak(self, nums: List[int]) -> int:
    left = 0
    right = len(nums) - 1
    while left < right:
        mid = (left + right) // 2
        if nums[mid] < nums[mid + 1]:
            left = mid + 1
        else:
            right = mid
    return left
```

Content Copyright Nanyang Technological University

Q4 (Navigator) – Answer

create variables left and right

while left less than right find mid

if mid less than mid+1, update left, otherwise update right

return left

```
def find_peak(self, nums: List[int]) -> int:
    left = 0
    right = len(nums) - 1
    while left < right:
        mid = (left + right) // 2
        if nums[mid] < nums[mid + 1]:
            left = mid + 1
        else:
            right = mid
    return left
```

Content Copyright Nanyang Technological University